

UNIVERSITATEA "ALEXANDRU-IOAN CUZA" DIN IAȘI

FACULTATEA DE INFORMATICĂ



LUCRARE DE LICENȚĂ

**Clasificarea genurilor muzicale pe baza
elementelor vocale**

propusă de

Andreea-Roxana Irimia

Sesiunea: iulie, 2021

Coordonator științific

Asist.dr. Croitoru Eugen

UNIVERSITATEA "ALEXANDRU-IOAN CUZA" DIN IAȘI

FACULTATEA DE INFORMATICĂ

Clasificarea genurilor muzicale pe baza elementelor vocale

Andreea-Roxana Irimia

Sesiunea: iulie, 2021

Coordonator științific

Asist.dr. Croitoru Eugen

Avizat,

Îndrumător Lucrare de Licență

Titlul, Numele și prenumele _____

Data _____ Semnătura _____

DECLARAȚIE privind originalitatea conținutului lucrării de licență

Subsemnatul(a)

domiciliul în

născut(ă) la data de, identificat prin CNP,
absolvent(a) al(a) Universității „Alexandru Ioan Cuza” din Iași, Facultatea de
..... specializarea, promoția
....., declar pe propria răspundere, cunoscând consecințele falsului în
declarații în sensul art. 326 din Noul Cod Penal și dispozițiile Legii Educației Naționale nr.
1/2011 art.143 al. 4 și 5 referitoare la plagiat, că lucrarea de licență cu titlul:

_____elaborată sub îndrumarea dl. / d-na
_____, pe care urmează să o susțină în fața
comisiei este originală, îmi aparține și îmi asum conținutul său în întregime.

De asemenea, declar că sunt de acord ca lucrarea mea de licență să fie verificată prin
orice modalitate legală pentru confirmarea originalității, consimțind inclusiv la
introducerea conținutului său într-o bază de date în acest scop.

Am luat la cunoștință despre faptul că este interzisă comercializarea de lucrări
științifice în vederea facilitării falsificării de către cumpărător a calității de autor al unei
lucrări de licență, de diploma sau de disertație și în acest sens, declar pe proprie

răspundere că lucrarea de față nu a fost copiată ci reprezintă rodul cercetării pe care am întreprins-o.

Data azi,

Semnătură student

DECLARAȚIE DE CONSIMȚĂMÂNT

Prin prezenta declar că sunt de acord ca Lucrarea de licență cu titlul „*Clasificarea genurilor muzicale pe baza elementelor vocale*”, codul sursă al programelor și celelalte conținuturi (grafice, multimedia, date de test etc.) care însoțesc această lucrare să fie utilizate în cadrul Facultății de Informatică.

De asemenea, sunt de acord ca Facultatea de Informatică de la Universitatea „Alexandru Ioan Cuza” din Iași, să utilizeze, modifice, reproducă și să distribuie în scopuri necomerciale programele-calculator, format executabil și sursă, realizate de mine în cadrul prezentei lucrări de licență.

Iași,

Absolvent

(semnătura în original)

Cuprins

Introducere	2
0.1 Motivație	3
Contribuții	4
1 Abordări anterioare	5
1.1 Rețele convoluționale pentru clasificarea sunetelor	5
1.2 Clasificarea genurilor muzicale	5
2 Tehnologii folosite	7
2.1 Spotify API	7
2.2 Librosa	7
2.3 Spleeter	7
2.4 Tensorflow	8
2.5 Sklearn	8
2.6 Flask	8
2.7 React	8
2.8 MongoDB	8
3 Construirea setului de date	10
3.1 Colectarea fișierelor audio	11
3.2 Izolarea elementelor vocale	11
3.3 Curățarea setului de date	11
3.3.1 Eliminarea porțiunilor de liniște	12
3.3.2 Identificarea și eliminarea fișierelor care conțin doar liniște	13
3.3.3 Identificarea și eliminarea segmentelor care conțin doar zgomote	14

4	Prelucrarea fişierelor audio	16
4.1	Spectrograme Mel	17
4.2	MFCC	19
5	Încărcarea datelor	21
5.1	Încărcarea întreg setului de date în memorie	22
5.2	Generatori	23
5.3	Experimente	24
6	Arhitectura reţelei	25
6.1	Experimente	25
6.1.1	CNN	25
6.1.2	LSTM	29
6.1.3	CNN-LSTM	31
6.1.4	Comparaţii	32
6.2	Selecţia genurilor	34
6.3	Distribuţia datelor	36
7	Sistemul de recomandare	38
7.1	Recomandare	38
7.2	Prezicerea genurilor muzicale	39
7.3	Experimente	40
	Concluzii	44
7.4	Direcţii viitoare	44
7.4.1	Setul de date	44
7.4.2	Arhitectura reţelei	44
7.4.3	Noi funcţionalităţi	44
	Bibliografie	45

Introducere

În ultimii ani, domeniul procesării semnalelor audio (*Audio Signal Processing*) se bucură de o popularitate crescută datorită multitudinii de aplicații practice existente precum anularea zgomotului (*noise cancelling*), procesarea vorbirii (*speech processing*), clasificarea sunetelor, dar în special aplicații în domeniul muzical (*Music Information Retrieval*) începând cu clasificarea genurilor muzicale sau a artiștilor pentru crearea de sisteme de recomandare sau identificarea încălcării drepturilor de autor prin recunoașterea segmentelor muzicale în videoclipuri, până la generarea unor segmente muzicale noi pornind de la anumite genuri sau artiști ¹.

În ceea ce privește problema clasificării genurilor muzicale, progresele sunt semnificative, luând în calcul un anumit grad de subiectivitate legat de apartenența melodiilor la anumite genuri [1]. Odată cu evoluția tehnologiei, dimensiunea datelor de intrare pentru algoritmi de clasificare nu mai este limitată la reprezentări de dimensiune redusă. Dacă la început experimentele se axau pe identificarea caracteristicilor de mici dimensiuni dar cu o putere discriminatoare mare prezente în segmentele muzicale (*zero crossing rate*, *MFCC mean*, *spectral flow mean*[2]) și aplicarea unor algoritmi precum KMeans, Knn, mașini cu vectori suport sau rețele neuronale cu un număr redus de straturi, acum se utilizează date de dimensiuni mari, chiar neprelucrate (forme de undă) ca date de intrare pentru rețele neuronale, de asemenea, de dimensiuni mari, care reușesc să extragă cu succes caracteristicile specifice ale fiecărui gen și să încadreze segmentele într-un gen pe baza acestor caracteristici [4, 5].

În ambele abordări descrise, partea instrumentală a segmentului muzical joacă un rol important, în special elementele de percuție care sunt bine sintetizate în reprezentări, fie ele de dimensiuni mici (*zero crossing rate*) sau de dimensiuni mari (spectrograme). În acest sens, există multe experimente care încearcă să rezolve problema de clasificare utilizând doar informații legate de partea instrumentală a segmentelor

¹<https://openai.com/blog/jukebox/>

muzicale[6]. În mod surprinzător, puține lucrări s-au concentrat doar pe aspectul vocal în această problemă de clasificare, cele existente utilizând doar reprezentări de dimensiuni mici și mașini cu vectori suport pentru clasificare[7] sau pe clasificarea artistilor, nu a genurilor muzicale[8].

Această lucrare abordează problema clasificării genurilor muzicale, încadrând segmente muzicale într-un anumit gen, utilizând doar elementul vocal. Lucrarea propune un clasificator ce combină o rețea neuronală convoluțională cu o rețea neuronală recurentă și are ca date de intrare spectograme Mel.

Aplicarea practică a lucrării este un sistem de recomandare sub forma unei aplicații web care, pe baza unui input vocal, încadrează vocea într-o serie de genuri pe baza rezultatelor clasificării segmentelor în care a fost împărțit inputul și sugerează melodii similare existente în setul de date.

0.1 Motivație

Motivația acestei lucrări este descoperirea informațiilor audio încapsulate în vocea unui artist care interpretează o anumită melodie. Fiecare gen muzical poate impune un anumit stil care poate fi identificat prin diferențele de ritm, inflexiuni sau modul de accentuare al versurilor. Astfel, analizând vocea unei persoane într-o anumită melodie, putem să extragem informații esențiale pentru a încadra vocea într-un anumit gen, dar și pentru a identifica voci similare.

Aplicabilitatea acestei idei este faptul că o persoană are posibilitatea să descopere artiști cu o voce similară cu cei pe care îi ascultă deja sau poate să descopere alți artiști cu o voce similară cu cea pe care o are, găsind astfel melodii pe care le-ar putea cânta sau inspirație.

Contribuții

Înainte de construirea rețelei, a fost nevoie de un set de date care să conțină informații prelucrate extrase din segmente audio ce conțin doar elemente vocale, izolate de partea instrumentală sau fișiere audio ce permit izolarea părții instrumentale de cea vocală și apoi prelucrarea părții vocale. Seturi de date deja existente nu au putut fi utilizate deoarece nu conțin fișierele audio, ci doar date prelucrate din care nu se pot izola elementele vocale sau, în cazul celor care conțin fișiere audio, nu au un număr suficient de date sau o diversitate destul de mare a genurilor muzicale. Așadar, a fost necesară construirea unui set de date etichetate format din fișiere audio ce conțin doar elementele vocale din melodiile originale.

A fost construit atât un pipeline de procesare a setului de date și salvarea lui sub forma unor reprezentări ușor de preluat ca input pentru rețeaua neuronală, cât și un pipeline de încărcare în memorie a acestor reprezentări. În ceea ce privește rețeaua neuronală, au fost făcute experimente cu mai multe tipuri de arhitecturi în urma cărora a fost selectată arhitectura cu cele mai bune rezultate, o rețea ce primește ca input spectrograme Mel, utilizează o rețea convoluțională pentru a reduce dimensionalitatea spectrogramei și a extrage caracteristicile ei esențiale, aceste caracteristici reprezentând inputul pentru o rețea recurentă care tratează aspectul temporalității spectrogramei.

În final, a fost construită aplicația web care permite vizualizarea predicțiilor pe date noi, în comparație cu segmentele din setul de date cu vectori de predicție similari.

Capitolul 1

Abordări anterioare

1.1 Rețele convoluționale pentru clasificarea sunetelor

Prin experimentele celor de la Google prezentate în [3] s-a demonstrat că rețelele de mari dimensiuni ce au la bază straturi convoluționale precum ResNet-50 [16] și AlexNet [17] au rezultate mai bune în comparație cu rețelele cu straturi dens conectate și au fost utilizate pentru clasificarea a până la 30 de mii de sunete prezente într-un set de date de 70 de milioane de videoclipuri.

1.2 Clasificarea genurilor muzicale

Odată cu creșterea popularității platformelor de streaming audio, problema clasificării genurilor muzicale a devenit, de asemenea, mai populară.

În [15] au fost comparate performanțele rețelelor convoluționale ce primesc ca input spectrograme, cu cele ale algoritmilor SVM care primesc ca input caracteristici extrase manual din semnalul audio și apoi s-a încercat combinarea acestora. Experimentele s-au bazat pe trei seturi de date diferite, rezultatele obținute pe două dintre acestea pentru combinația dintre CNN și SVM depășind rezultatele existente precendent în literatură.

O altă abordare des întâlnită este combinarea rețelelor convoluționale cu rețele neuronale recurente. Experimentele au demonstrat că un CNN combinat cu o rețea recurentă are rezultate mai bune decât rețelele convoluționale simple [4] [5].

În ceea ce privește clasificarea genurilor muzicale pe baza elementelor vocale, cele mai multe experimente s-au axat pe clasificarea artiștilor, nefiind cazul de o separare

a surselor sonore pentru a izola vocea. Un experiment care se concentrează exclusiv pe elementele vocale este prezentat în [7]. Cu toate acestea, utilizează caracteristici de dimensiuni mici extrase din semnalul audio și un algoritm de clasificare SVM, iar experimentele prezentate anterior demonstrează faptul că rezultate mai bune pot fi obținute cu rețele neuronale și reprezentări de dimensiuni mai mari ale semnalelor audio precum spectrograme sau coeficienți MFCC.

Capitolul 2

Tehnologii folosite

2.1 Spotify API

API-ul oferit de Spotify [23] a fost utilizat atât pentru construirea setului de date, cât și pentru a putea include segmentele similare cu inputul utilizatorului embedded în aplicația Web. Motivul principal pentru care a fost ales acest API este funcționalitatea de recomandare care permite ca, printr-un request în care este specificat un anumit gen muzical, să se obțină o listă de albume și, ulterior, segmente în format audio care se încadrează în acel gen. De asemenea, considerând diversitatea genurilor muzicale oferite, acest API rezolvă ambele probleme ce au intervenit în obținerea unui set de date.

2.2 Librosa

Librosa [24] este un pachet care pune la dispoziție funcționalități de analiză și prelucrarea a fișierelor audio. A fost folosit în această lucrare pentru prelucrarea datelor, extragerea reprezentărilor necesare rețelei neuronale, dar și pentru extragerea de informații în vederea curățării setului de date.

2.3 Spleeter

Spleeter [25] este o bibliotecă construită cu Tensorflow care folosește modele pre antrenate pentru separarea surselor audio. În aceasta lucrare, a fost utilizată pentru a izola elementele vocale de instrumental. O bibliotecă alternativă a fost Open-Unmix

[26], dar Spleeter a fost aleasă deoarece are performanțe mai bune pe partea de separare vocală [9].

2.4 Tensorflow

Principalele funcționalități oferite de Tensorflow [27] folosite în această lucrare sunt Keras, necesară pentru construirea și antrenarea rețelei neuronale, tf.data API [28], utilizată pentru crearea unui input pipeline pentru rețeaua construită și Tensorboard, necesar pentru vizualizarea unor metrici ale procesului de antrenare.

2.5 Sklearn

Amestecarea datelor, generarea matricelor de confuzie și calcularea celor mai apropiați vectori de predicție se realizează utilizând funcționalitățile bibliotecii Sklearn [29].

2.6 Flask

Pentru implementarea serverului web responsabil cu primirea fișierelor audio, procesarea lor și generarea de predicții a fost ales framework-ul Flask [30], în special datorită arhitecturii mai simple, în comparație cu alte soluții existente, care se pliază pe nevoile curente ale aplicației.

2.7 React

Pentru implementarea aplicației frontend, a fost utilizat React [31] deoarece facilitează dezvoltarea mai rapidă a aplicației prin utilizarea componentelor predefinite și, în comparație cu alternativa Angular, are avantajul simplității.

2.8 MongoDB

Am utilizat un cluster de MongoDB [32] pentru a salva informații adiționale despre melodiile din setul de date precum numele melodiei, albumul din care face parte și url-ul de la care poate fi descărcată. Principalul motiv pentru care am optat pentru un

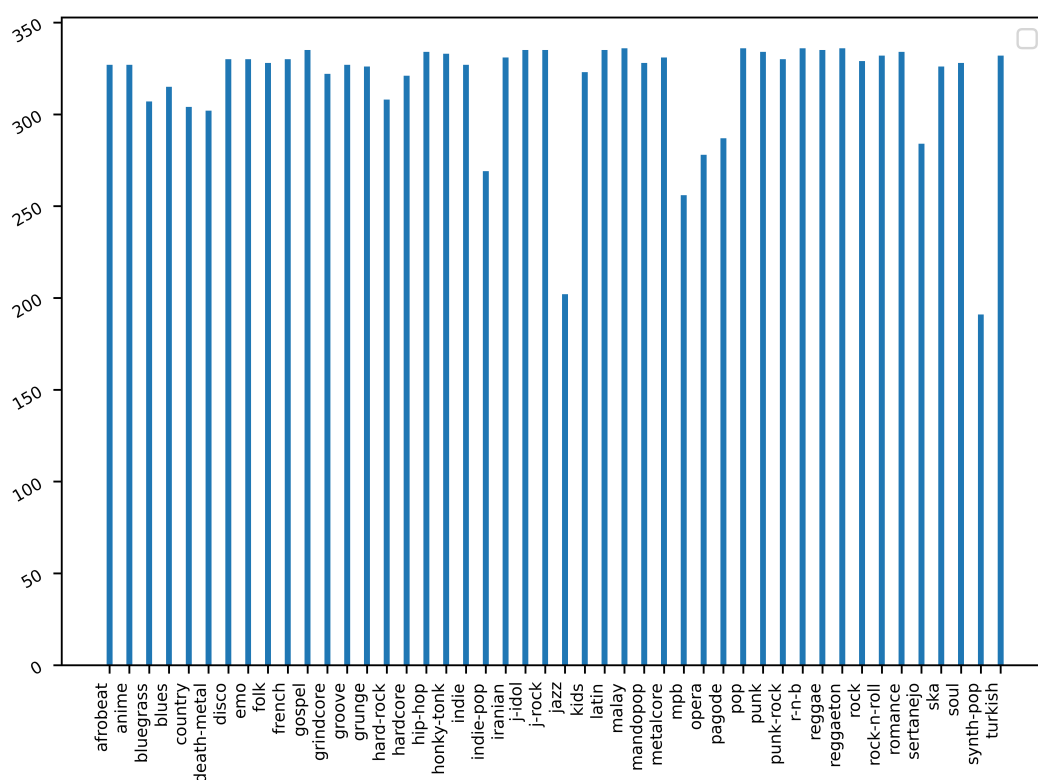
serviciu de baze de date în Cloud este ușurința prin care datele pot fi accesate de pe mai multe dispozitive. Astfel, melodiile au putut fi descărcate în paralel, iar existența datelor duplicate a fost evitată prin verificarea existenței în baza de date care ține evidența tuturor descărcărilor.

Capitolul 3

Construirea setului de date

Setul de date final conține 14.500 de fișiere audio distincte, împărțite pe 46 de genuri muzicale. Dacă inițial numărul de fișiere era egal pentru fiecare gen, în urma curățării setului de date, distribuția nu a mai rămas egală, acest aspect fiind tratat în subcapitolul 6.3 Distribuția menționată poate fi observată în figura 3.1.

Figura 3.1: Distribuția fișierelor per genuri muzicale



În continuare sunt prezentate, în mai multe detalii, etapele construirii setului de date.

3.1 Colectarea fișierelor audio

Pentru colectarea fișierelor audio, a fost utilizat API-ul pus la dispoziție de Spotify. Printr-un request în care este specificat genul dorit, se obține o listă de albume care se încadrează în acest gen. Pentru fiecare album, se face un request prin care, specificând id-ul albumului se obține lista de cântece din acel album. Astfel, avem acces la id-ul fiecărei melodii, genul în care se încadrează și un url de vizualizare de unde poate fi descărcat un segment de 30 de secunde din melodie. Pentru a reține detaliile despre melodiile deja colectate, gen, id-ul melodiei, id-ul albumului din care provine și url-ul pentru previzualizare, s-a utilizat un cluster de MongoDB. Aceste detalii adiționale vor fi utilizate ulterior pentru realizarea experimentelor.

Odată ce aceste informații au fost obținute, pentru fiecare melodie care nu este deja înregistrată în baza de date, segmentul audio va fi descărcat și procesat.

3.2 Izolarea elementelor vocale

Pentru a izola elementele vocale de partea instrumentală am utilizat Spleeter, un proiect de separare a surselor audio lansat de Deezer în 2019 ¹, în varianta de API python, pentru a integra ușor acest pas direct în pipeline-ul de construire a setului de date. Astfel, imediat după descărcare, melodia este automat împărțită în două fișiere, voce și instrumental.

Pentru a realiza acest lucru, Spleeter utilizează o rețea U-Net [18], un auto-encoder convolutional cu conexiuni de tip skip, modificată pentru a procesa date de tip 3D (timp, frecvență și canale audio), preantrenată pe un set de date de aproape două sute de ore de înregistrări.[10]

3.3 Curățarea setului de date

În urma separării audio, au fost necesari pași adiționali de procesare a datelor deoarece au fost identificate două probleme.

Prima problemă constă în pauzele vocale din melodiile descărcate ce în urma separării au devenit segmente de liniște care nu dețin informații în vederea clasificării,

¹<https://deezer.io/releasing-spleeter-deezer-r-d-source-separation-engine-2b88985e797e>

reprezintă elemente de zgomot în antrenarea modelului și, așadar, este necesară eliminarea lor.

A doua problemă constă în elemente de instrumental care nu au fost bine separate de modelul preantrenat utilizat. Problema apare preponderent în cazul genurilor *hard-rock*, *heavy metal*, *death-metal* unde apar solo-uri de chitară electrică, iar notele înalte ale chitarei nu sunt distinse de notele înalte ale artistului, dar și în cazul melodiilor care conțin instrumente de suflat precum saxofonul deoarece are spectrul sonor mai apropiat de cel al vocii umane, după cum se poate observa în figura 3.2

În continuare sunt descriși pașii implementați pentru a rezolva aceste probleme.

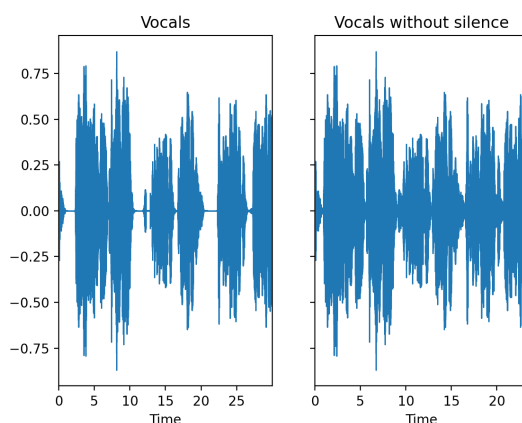
Figura 3.2: Comparatie a spectrogramelor pentru nota E4



3.3.1 Eliminarea porțiunilor de liniște

Pentru a rezolva prima problemă am folosit funcția *librosa.effects.split*². Această funcție separă semnalul audio în segmente care nu conțin liniște. După apelarea funcției, segmentele rezultate vor fi concatenate, rezultând un segment audio fără pauze de liniște după cum poate fi observat în figura 3.3

Figura 3.3: Waveform înainte și după curățarea porțiunilor de liniște



²[librosa.effects.split](#)

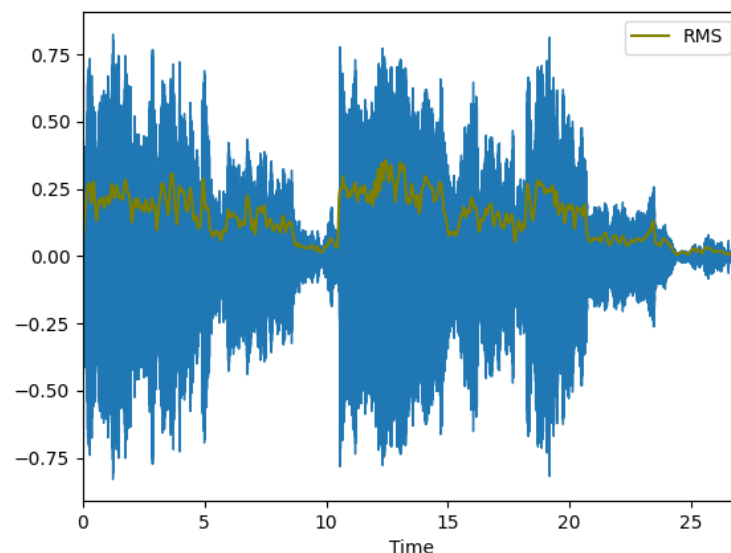
Noțiunea de liniște în acest caz este considerată ca fiind o secvență cu un număr de decibeli mai mic decât un threshold (threshold-ul utilizat a fost 35 de decibeli) sub numărul de decibeli de referință. Numărul de decibeli de referință este numărul maxim de decibeli din semnalul sonor. Această procesare a fost integrată în pipeline-ul de construire a setului de date, curățarea segmentelor de liniște fiind realizată imediat după separarea segmentului inițial în voce și instrumental. De asemenea, dacă după eliminarea liniștei semnalul audio rezultat nu conține cel puțin numărul de secunde per segment necesar la procesare, melodia nu va fi inclusă în setul de date deoarece nu oferă destule informații vocale.

3.3.2 Identificarea și eliminarea fișierelor care conțin doar liniște

A fost menționat faptul că numărul de decibeli de referință este numărul maxim de decibeli din semnalul sonor. Acest aspect este important deoarece, în cazul segmentelor care după separare conțin doar liniște, numărul de decibeli de referință este mic iar nicio valoare nu va fi mai mică decât un threshold sub numărul de referință.

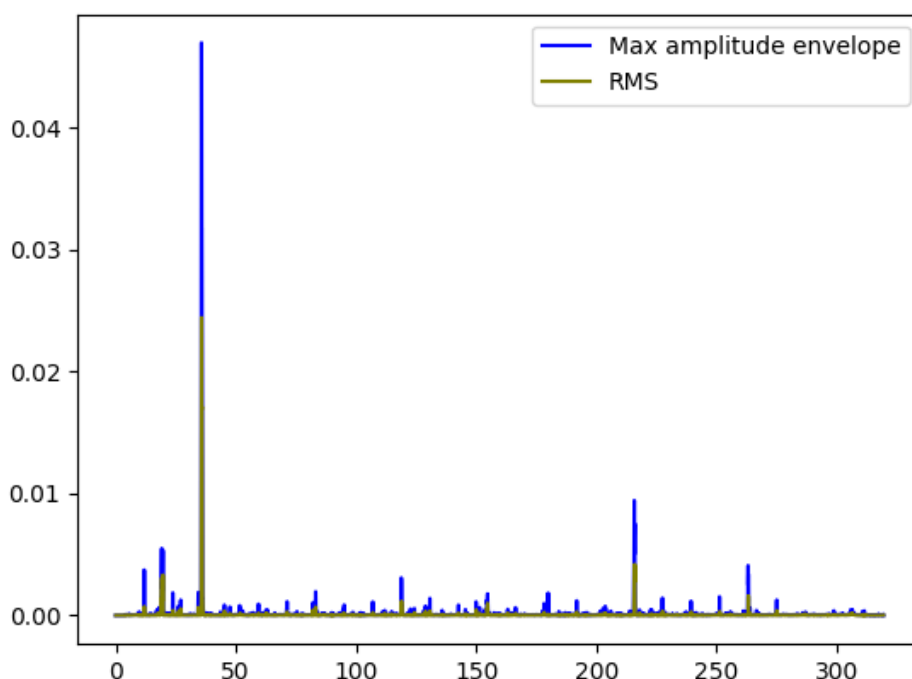
O consecință a acestui aspect este faptul că nu se va identifica nicio porțiune de liniște. Pentru a rezolva acest caz, se calculează media pătratică a waveform-ului (RMS) pentru fiecare frame și se compară această valoare cu un threshold (0.05). O reprezentare a valorilor mediilor patratice pentru frame-uri în comparație cu valorile waveform-ului poate fi observată în figura 3.4

Figura 3.4: Valorile RMS



RMS a fost ales în favoarea amplitudinii maxime deoarece evidențiază intensitatea semnalului audio pentru o perioadă mai lungă de timp. Valoarea amplitudinii maxime pentru un punct în timp izolat nu este reprezentivă pentru întregul frame și de aceea s-a optat pentru media amplitudinilor maxime, modul în care acestea evoluează fiind și mai apropiat de modul în care oamenii percep schimbările de intensitate. Diferența poate fi observată în figura 3.5

Figura 3.5: Valorile RMS

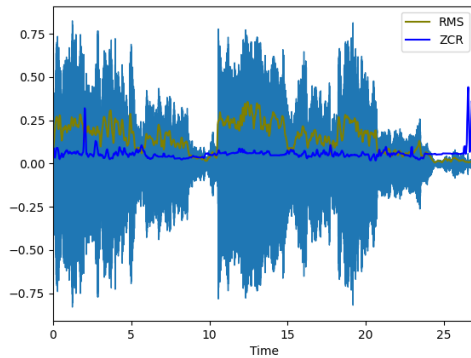


3.3.3 Identificarea și eliminarea segmentelor care conțin doar zgomote

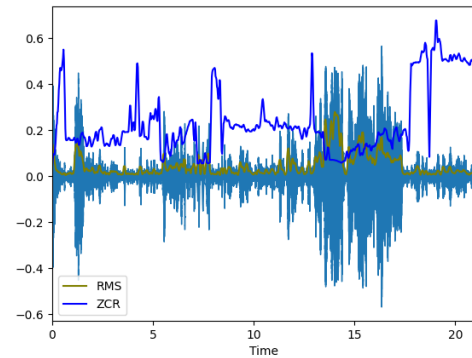
În cazul elementelor de instrumental care nu au fost bine separate de modelul preantrenat utilizat, chiar dacă nu sunt prezente elemente de voce, segmentul nu este identificat ca unul ce conține doar liniște deoarece instrumentalul care nu a fost separat cu succes are o intensitate destul de mare. Un segment care se încadrează în această categorie poate fi ascultat aici.

Pentru a identifica acest tip de segmente s-a ținut cont de faptul că pentru zgomote, perioada de traversare la nivel de zero (*Zero Crossing Rate*) este mai mare decât

cea a vocii umane. Așadar se poate deduce că, în cazul unui segment care conține doar zgomote, dacă valoarea maximă pentru perioada de traversare la nivel de zero este mai mare decât un anumit threshold (0.5), iar valoarea RMS maximă este mai mică decât un anumit threshold (0.4), atunci conține doar zgomote și va fi eliminat din setul de date. Comparăția dintre un segment care conține elemente vocale și un segment care conține doar zgomote poate fi observată în figura 3.6



(a) RMS și ZCR pentru voce



(b) RMS și ZCR pentru zgomot

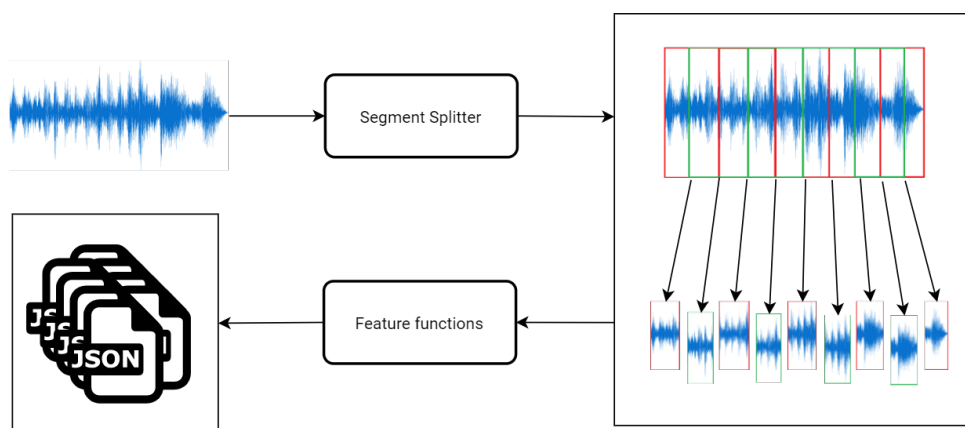
Figura 3.6: Comparăție între RMS și ZCR pentru voce și zgomot

Capitolul 4

Prelucrarea fișierelor audio

Odată ce toate fișierele audio au fost descărcate, separate în voce și instrumental iar fișierele ce conțin vocea au fost curățate prin procesul descris în capitolul anterior, a fost nevoie de un pipeline de extragere a datelor din fișierele audio. Pipeline-ul are scopul de a face procesul de extragere destul de flexibil încât să permită rularea de experimente pentru segmente de diferite dimensiuni și diferite tipuri de reprezentări ale informațiilor audio. Pașii implementați pot fi observați în figura 4.1

Figura 4.1: Procesul de extragere al informațiilor din fișierele audio



Pipeline-ul primește ca input directorul în care se află fișierele audio, directorul sau fișierul în care vor fi salvate informațiile extrase, în funcție de forma sub care dorim să le salvăm, o listă de instanțe ale clasei *Feature Function*, o instanță a clasei *Segment Splitter* și, opțional, un procent în cazul în care se dorește procesarea unui subset al setului de date, opțiune utilă în procesul de testare al funcționalităților pe diferite mașini.

Clasa *Segment Splitter* se ocupă de împărțirea semnalului audio în mai multe segmente, de dimensiuni fixe. La crearea unei instanțe pot fi specificate numărul de secunde pe care îl vor avea segmentele și o listă de offset-uri de la care să înceapă sepa-

rarea. De exemplu, după cum am exemplificat și în figura 4.1 dacă un offset nu este specificat, în urma separării vor rezulta doar segmentele reprezentate cu roșu, iar dacă specificăm un offset, vor rezulta și segmentele reprezentate cu verde. Astfel, se asigură faptul că toate datele de input pentru rețea au aceeași dimensiune prin prisma faptului că segmentele rezultate au lungimi egale. De asemenea, specificând unul sau mai multe offset-uri, dimensiunea setului de date crește considerabil.

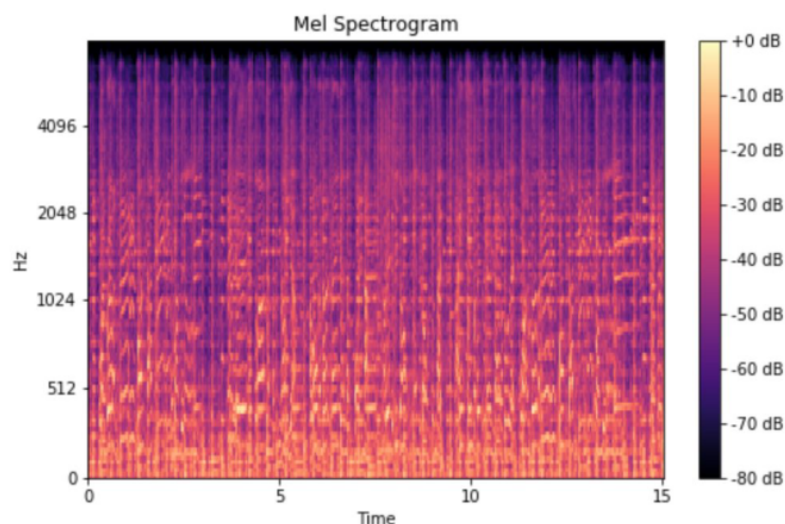
Clasa *Feature Function* modelează un tip de informație pe care vrem să o extragem din segmentele audio. O instanță primește o funcție care, pentru un segment audio va returna o informație extrasă din acel segment, parametrii adiționali cu care va fi apelată funcția specificată și un nume prin care vom identifica informația extrasă. Această abordare asigură elementul de flexibilitate al pipeline-ului. Pentru un tip nou de informație este necesară doar definirea unei noi funcții.

Informațiile extrase pentru un segment pot fi salvate individual ca JSON sau Tensors serializați binar sau pot fi adăugate ca JSON într-un fișier ce conține informațiile extrase din toate segmentele. Gama largă de reprezentări permite experimentarea cu diferite tipuri de încărcare a datelor și diferite tipuri de inputuri în antrenarea rețelei.

În continuare sunt descrise două tipuri de informații care au fost extrase din segmentele audio.

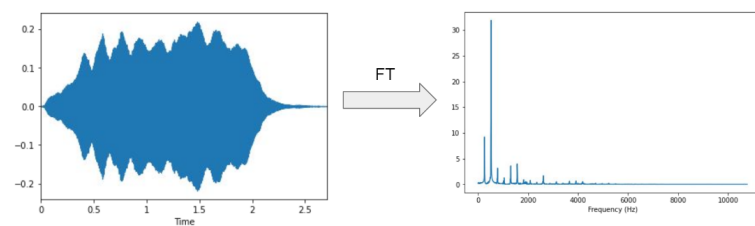
4.1 Spectrograme Mel

Figura 4.2: Spectrograma mel



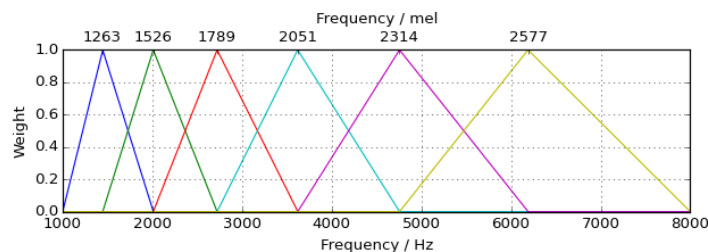
O spectrogramă este o metodă de vizualizare a evoluției spectrului de frecvențe și intensitatea lor în timp, care se obține aplicând o transformare Fourier de scurtă durată asupra forme de undă. Transformarea Fourier de scurtă durată se obține aplicând transformări Fourier discrete pe ferestre de mici dimensiuni ale forme de undă care de obicei se suprapun [19]. Transformarea Fourier discretă face trecerea de la domeniul de timp la domeniul de frecvență, descompunând semnalul audio în frecvențele ce îl compun, comparând semnalul cu sinusoidale diferite la diferite faze de undă după cum se poate observa în figura 4.3.¹

Figura 4.3: Trecerea de la domeniul de timp la domeniul de frecvență



În urma aplicării unei transformări Fourier de scurtă durată obținem o evoluție în timp a frecvențelor prezente în semnalul audio. Atât amplitudinea cât și frecvențele rezultate sunt reprezentate liniar în urma aplicării acestei transformări. Cu toate acestea, modul în care oamenii percep aceste valori este logaritmice. De aceea, pentru a apropia această reprezentare de modul în care sunetele sunt percepute, amplitudinea este convertită în decibeli, iar frecvența este convertită conform scării Mel $m = 2595 \cdot \log(1 + \frac{f}{500})$, unde f este frecvența exprimată în Hertz și m este frecvența exprimată în Mel. Se plasează pe intervalul dintre cea mai joasă frecvență Mel și cea mai înaltă frecvență Mel un număr de puncte echidistante egal cu numărul de filtre mel dorite (128) care vor fi centrul filtrelor triunghiulare Mel 4.4².

Figura 4.4: Filtre Mel



¹Demystifying the Fourier Transform

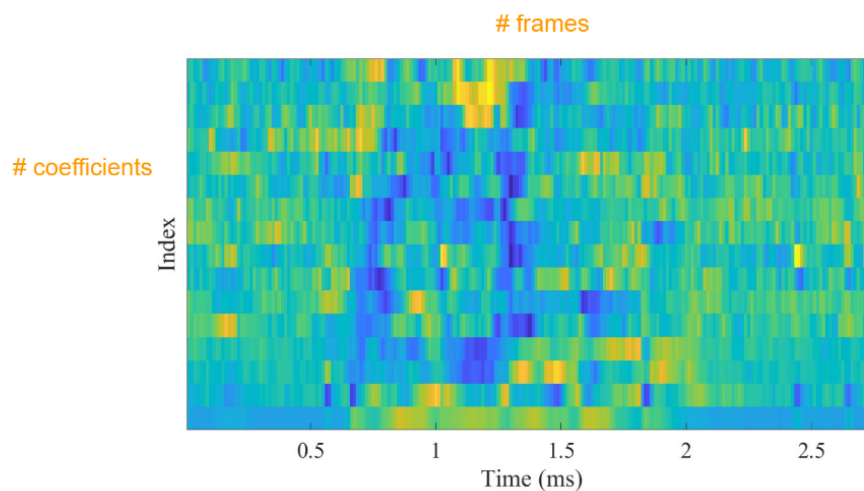
²<http://sigggie.github.io/pyfilterbank/melbank.html>

Pentru a obține spectrograma Mel, spectrograma rezultată în urma transformării Fourier de scurtă durată cu amplitudinea exprimată în decibeli este înmulțită cu filtrele Mel calculate.

Această reprezentare a fost aleasă deoarece este apropiată din punct de vedere perceptual de modul în care urechea umană procesează sunetele, și are rezultate mai bune în literatură în problema clasificării genurilor muzicale[4], sau clasificării sunetelor în general[3] decât spectrogramele simple sau MFCC atunci când rețeaua este una convoluțională.

4.2 MFCC

Figura 4.5: MFCC

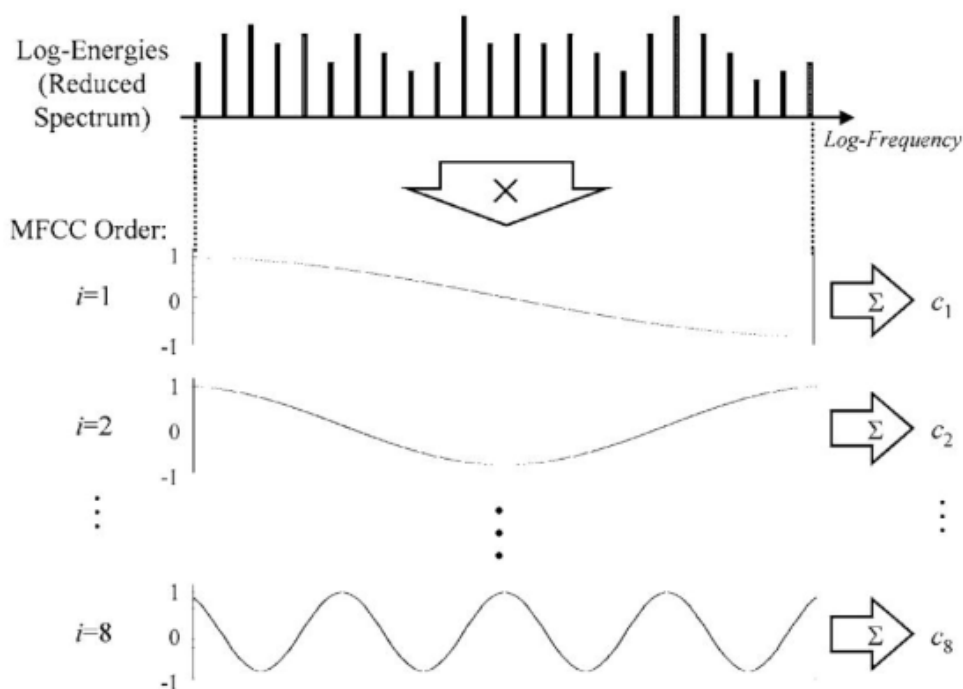


Extragerea coeficienților MFCC 4.5 (Mel-Frequency cepstral coefficients) se obține realizând aceiași pași descriși în extragerea unei spectrograme, o transformare Fourier de discretă, convertirea amplitudinii în decibeli și aplicarea bancului de filtre Mel, urmați de logaritmare a spectrului Mel, aplicarea unei transformări cosinusoidale discrete pentru fiecare fereastră a semnalului audio și calcularea derivatelor de ordinul întâi și ordinul al doilea, numite coeficienți delta, respectiv coeficienți delta delta[20].

Transformarea cosinusoidală discretă este un proces similar cu transformare Fourier discretă, diferența fiind faptul că transformarea cosinusoidală discretă returnează valori din mulțimea numerelor reale în timp ce transformarea Fourier discretă poate returna valori complexe. Astfel, se calculează similaritatea dintre spectrul Mel logarit-

mic și mai multe funcții cosinusoidale ce corespund diferitelor frecvențe 4.6 ³, această similaritate rezultată reprezentând un coeficient MFCC.

Figura 4.6: Transformarea cosinusoidală discretă



În timp ce coeficienții MFCC oferă informații relevante despre formanții și fone-
mele prezente în semnalul sonor pentru recunoașterea timbrului și *speech recognition* în
general, coeficienții delta și delta delta oferă informații temporale în legătură cu ritmul
vorbirii. Datorită acestor aspecte, coeficienți MFCC sunt frecvenți utilizați în proble-
mele de *speech recognition*, principalul avantaj fiind dimensiunea redusă în comparație
cu spectrogramele Mel. Astfel, în funcție de arhitectura rețelei, a fost preferată utili-
zarea coeficienților MFCC în favoarea spectrogramei Mel tocmai datorită dimensiunii
reduse, în special în cazurile în care nu s-au utilizat straturi convoluționale.

³Mel-Frequency Cepstral Coefficients Explained

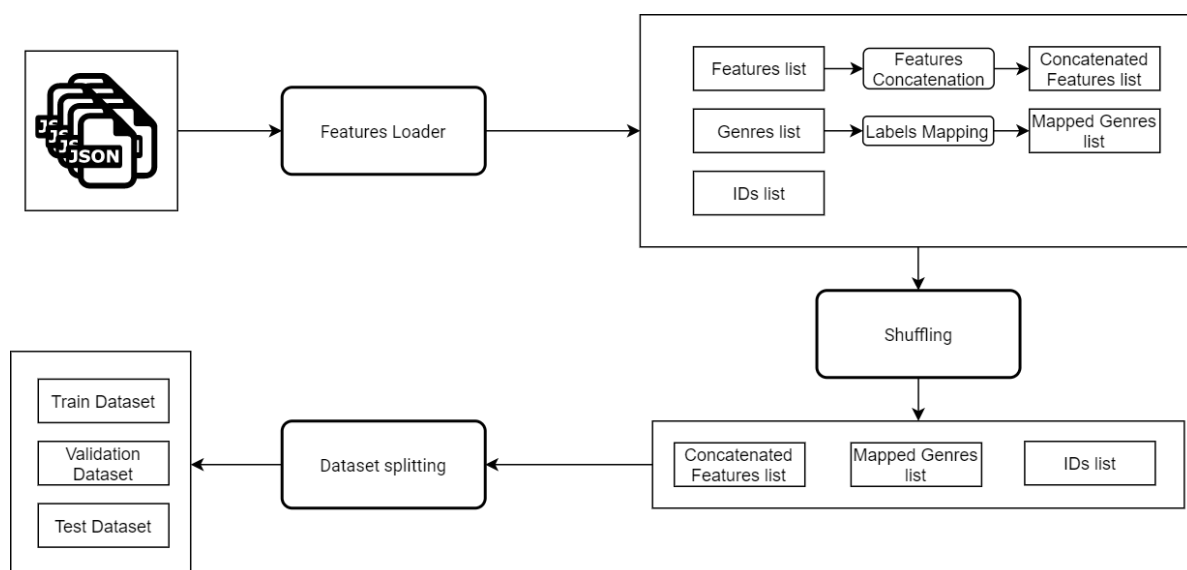
Capitolul 5

Încărcarea datelor

Având în vedere formatele diferite în care informațiile extrase din fișierele audio pot fi salvate pe disc, același nivel de flexibilitate este necesar și la încărcarea acestor date în memorie. Nevoia de a implementa diferite moduri de încărcare pentru tipuri diferite de formate ale datelor a fost un rezultat al capabilităților mașinilor pe care rețeaua a fost antrenată. În funcție de aceste capabilități, dar și în funcție de dimensiunea informațiilor extrase, datele puteau sau nu să fie încărcate în totalitate în memoria RAM. Această problemă a fost rezolvată prin experimentarea cu diferite tipuri de generatoare care încarcă datele de pe disc în timpul antrenării rețelei, scopul final fiind găsirea unei metode care minimizează timpul de antrenare.

O schemă generală a încărcării datelor în memorie și pregătirea inputurilor de antrenare, validare și test ale rețelei este prezentată în figura 5.1

Figura 5.1: Procesul de încărcare al datelor în memorie



După cum a fost menționat, au existat două abordări, în funcție de dimensiunea datelor și memoria RAM disponibilă. În continuare sunt descrise particularitățile acestor abordări.

5.1 Încărcarea întreg setului de date în memorie

În acest caz, pipeline-ul primește ca input o instanță a unei clase derivate din clasa de bază *Features Loader* și, opțional, un state random ce va fi utilizat la amestecarea datelor.

Clasa *Features Loader* se ocupă de încărcarea informațiilor din fișiere și extragerea genurilor muzicale. În funcție de clasa concretă specificată, informațiile sunt încărcate din fișiere JSON, deserializate ca Tensori din fișiere binare, sau citite ca JSON linie cu linie dintr-un fișier, iar genurile sunt extrase din numele fișierelor, respectiv din obiectul JSON citit. La instanțiere se specifică calea directorului sau a fișierului în care se află datele și un subset al genurilor din setul de date. Opțiunea de a specifica un subset de genuri este utilizată în procesul ulterior de selecție a genurilor muzicale dezvoltat în secțiunea 6.2. În urma apelării funcției de încărcare se obțin trei liste ce corespund informațiilor extrase din fișierele audio prezente în setul de date, genurilor și id-urilor corespunzătoare acestor informații.

Fiecărui gen distinct încărcat îi este atribuit un număr întreg iar fiecare gen din listă este înlocuit cu numărul ce îi corespunde. În cazul informațiilor extrase din fișierele audio, dacă mai multe tipuri de informații sunt prezente, acestea vor fi concatenate.

Setul de date obținut va fi amestecat și împărțit în trei seturi de antrenare, test și validare conform unor procente ce pot fi specificate. Seturile de date rezultate pot fi *NumPy arrays* sau *tensorflow.data.Dataset*¹.

Cu toate că utilizarea *tensorflow.data.Datasets* în favoarea *NumPy arrays* rezultă într-o antrenare mai rapidă a rețelei, în funcție de dimensiunea datelor, nu este posibilă alocarea memoriei pentru crearea obiectelor *tensorflow.data.Dataset*. În aceste cazuri, au fost folosiți *NumPy arrays* ca input pentru rețea sau generatorii.

¹`tensorflow.data.Dataset`

5.2 Generatori

O soluție a problemei de memorie este utilizarea obiectelor *tensorflow.data.Dataset* construite peste generatori. Astfel, setul de date nu mai este încărcat în totalitate în memorie. Când se folosește această abordare pipeline-ul primește un obiect al clasei *File Names Features Loader*, derivată din clasa *Features Loader* care încarcă doar numele tuturor fișierelor din setul de date și genurile corespunzătoare. Procesul de amestecare, înlocuire a genurilor cu numere întregi și separare în date de antrenare, test și validare este același. Diferența constă în faptul că, după separarea datelor, sunt instanțiate trei generatoare ce se vor ocupa cu citirea informațiilor din fișier. Sunt create trei obiecte *tensorflow.data.Dataset* pe baza generatoarelor, utilizând funcția *from_generator*².

tensorflow.data.Dataset sunt optimizate pentru execuția pe GPU și permit utilizarea mecanismului de *prefetch* care permite, printr-un fir de execuție ce rulează în fundal, aducerea în memorie a datelor înainte ca ele să fie necesare, eliminând apariția de *bottlenecks*. Astfel, utilizarea *tensorflow.data.Dataset* este mai eficientă decât antrenarea cu generatori creați ca input direct al rețelei. Datorită faptului că utilizarea *tensorflow.data.Dataset* îmbunătățește timpul de antrenare al rețelei în comparație cu *NumPy arrays*, în cazurile în care dimensiunea memoriei RAM permitea acest lucru, s-au utilizat generatori care încărcău toate datele în memorie și le returnau pe rând utilizând *tensorflow.data.Dataset*. În acest mod, a fost rezolvată problema spațiului, menționată anterior, atunci când se alocă obiecte *tensorflow.data.Dataset* direct pentru listele încărcate în memorie.

În continuare se pot observa rezultatele experimentelor atunci când s-au utilizat modurile descrise de încărcare a datelor.

²`tensorflow.data.Dataset.from_generator`

5.3 Experimente

Statisticile legate de timpul de antrenare al rețelei au fost obținute utilizând Tensorboard ³. Comparațiile au fost facute pentru *tensorflow.data.Dataset* peste generatoare cu date sub forma de *Tensors* care încarcă datele progresiv (GS) sau care încarcă inițial toate datele în memorie (GP) și *NumPy arrays*. Rezultatele sunt prezentate în tabelul 5.1

Metodă	Timp mediu / step (ms)	Timp de execuție GPU (ms)	Încărcare date în GPU (ms)	Altele (ms)
GS	56.7	19.3	30.1	6.4
GP	23.0	17.7	0.1	5.2
NumPy	24.9	18.7	0.1	6.1

Tabela 5.1: Comparație între metodele de încărcare ale datelor

³<https://www.tensorflow.org/tensorboard>

Capitolul 6

Arhitectura rețelei

6.1 Experimente

În această secțiune sunt descrise diferite tipuri de rețele utilizate și arhitecturile utilizate, comparând rezultatele obținute și motivând alegerea arhitecturii finale.

6.1.1 CNN

Rețelele neuronale convoluționale (CNNs) sunt rețele neuronale utilizate, de obicei, pentru procesarea imaginilor. Prima aplicație ce are la bază rețelele convoluționale moderne [11] a demonstrat faptul că o astfel de rețea poate recunoaște cifrele scrise de mână din setul de date MNIST cu o rată de eroare de doar 0.7% (Boosted LeNet-4), un rezultat semnificativ mai bun decât alte rețele ce nu aveau la bază straturi convoluționale și, de asemenea, cu un număr mai mic de parametri ce pot fi antrenați.

Ideea de bază a acestui tip de rețea este emularea modului în care oamenii procesează imaginile, prin identificarea unor structuri simple care agregate, definesc structuri mai complexe iar în final obiecte. Acest proces se bazează pe două elemente fundamentale, convoluția și agregarea (pooling).

Convoluția

Un strat convoluțional are la bază unul sau mai multe nuclee (kernels). Fiecare nucleu este o matrice, bidimensională sau tridimensională, în funcție de numărul de canale al imaginii. Se calculează un produs scalar dintre nucleu și o submatricea imaginii de aceeași dimensiune cu cea a nucleului, rezultatul produsului fiind salvat în celula corespunzătoare celulei care se află în centrul submatricii selectate. Procesul

este ilustrat în figura 6.1¹. Pentru a calcula produsul scalar și pentru elementele de pe marginea matricii, în cazul în care nu se dorește reducerea dimensiunii, matricea va fi bordată.

Figura 6.1: Produsul scalar dintre submatrice și nucleu

Image						Kernel			Output					
5	2	3	1	2	4	1	0	0						
2	4	1	0	3	1	2	1	0		?				
5	1	0	2	8	3	1	0	-1						
0	2	1	5	2	4									
2	7	0	0	2	1									
1	3	2	8	7	0									

Nucleul se deplasează cu un anumit număr de poziții (strides) pe verticală și pe orizontală. Acest factor poate modifica, din nou, dimensiunea rezultatelor. Fiecare nucleu din stratul convoluțional va genera ca rezultat o matrice bidimensională. Prin antrenare, nucleele acționează ca filtre ce vor fi modificate pentru a identifica structurile de baza ale imaginii, linii și curbe în primele straturi convoluționale și structuri mai complexe la adâncimi mai mari.

După un strat convolutional, se aplică o operație de agregare (pooling).

Pooling

Pooling-ul este o operație ce are ca scop reducerea dimensiunii reprezentării. Similar cu procesul de convoluție, se stabilește o dimensiune de pooling și un stride. Diferența este faptul că, operația nu depinde de parametri ce pot fi antrenați, este doar un calcul asupra elementelor dintr-o submatrice cu aceeași dimensiune ca dimensiune de pooling stabilită. Această operație poate fi selectarea elementului maxim din submatrice (Max Pooling), calcularea mediei, calcularea mediei ponderate în funcție de distanța de la centrul matricii, etc.

¹How does a convolutional neural network work

Procesul de Max Pooling pentru o dimensiune de (2, 2) și un stride de 2 este ilustrat în figura 6.2

Figura 6.2: Max Pooling pentru dimensiunea (2, 2) și un stride de 2

Input				Output	
-1	2	0	2	18	10
3	18	10	-3		
2	12	5	2		
1	3	7	4		

Batch Normalization

Batch Normalization este un proces prin care input-urile straturilor rețelei sunt normalizate, rezultatul fiind o antrenare mai stabilă și rapidă a rețelei.

Normalizarea se obține calculând media batch-ului, $\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$, și deviația standard, $\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$, unde m este numărul de elemente din batch. Se calculează $\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$, iar pe baza acestor valori se calculează noile valori normalizate, $y_i = \gamma \hat{x}_i + \beta$, unde γ și β sunt parametrii ce pot fi antrenați. [12]

Arhitectura CNN implementată

Având în vedere că spectrogramele Mel și coeficienții MFCC pot fi considerate reprezentări similare cu o imagine cu un singur canal (grayscale), ce mențin aspectele de structură prezente în imagini, rețelele convoluționale sunt populare nu doar în procesarea imaginilor, ci și în procesarea audio. Arhitectura rețelei convoluționale implementate are 3 straturi convoluționale cu 32 de nuclee de dimensiunea (3, 3). Fiecare strat convoluțional este urmat de un strat de Max Pooling cu dimensiunile (3, 3), (3, 3), respectiv (2, 2) apoi de un strat de Batch Normalization. Output-urile sunt aplatizate și introduse ca input într-un strat dens conectat cu 64 de neuroni, urmat de stratul de output.

Dropout

Dropout-ul este un procedeu prin care, în timpul antrenării, un procent din neuroni rețelei dintr-un anumit layer sunt ignorați, alături de conexiunile cu ceilalți neuroni. Acești neuroni sunt selectați random la fiecare iterație. Rezultatul este prevenirea fenomenului de overfitting deoarece devine imposibil ca rețeaua să devină dependentă de un subset al neuronilor, atunci când aceștia pot deveni inactivi. Astfel, este încurajată generalizarea. [13]

Din acest motiv, între straturile dens conectate am utilizat un strat de Dropout de 0.3.

ReLU

Funcțiile de activare pentru straturile convoluționale și primul strat dens sunt ReLU, $f(x) = \max(0, x)$ deoarece previne problema de dispariție a gradientului (vanishing gradient) și are o complexitate computațională scăzută.

Softmax

Ultimul strat este unul dens conectat cu un număr de neuroni egal cu numărul distincte de genuri și activare Softmax. Fiecare valoare a output-ului va reprezenta un procent, sugestiv pentru gradul în care un anumit gen se regăsește într-un segment.

Adam

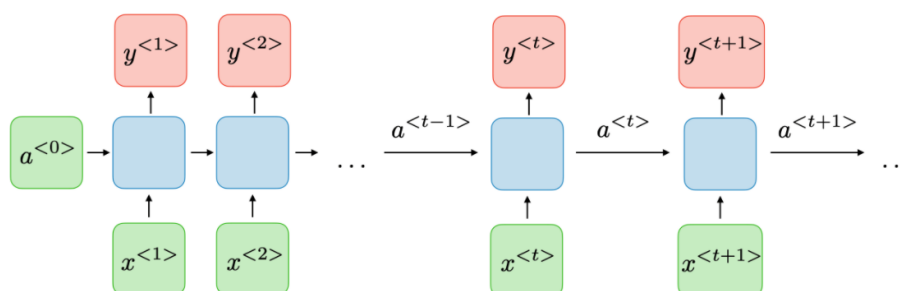
Optimizatorul utilizat a fost Adam cu o rată de învățare de 0.005 deoarece s-a demonstrat că are rezultate mai bune în practică decât stochastic gradient descent [14].

6.1.2 LSTM

Rețele neuronale recurente

Rețelele neuronale recurente sunt un tip de rețele neuronale în care conexiunile dintre noduri formează un graf orientat în care informația circulă în sensul axei temporale, prin existența unor stări și inputuri ascunse. [21] Structura generală a unei rețele recurente este ilustrată în figura 6.3 ².

Figura 6.3: Arhitectura tradițională unei rețele neuronale recurente



$a^{<t>} = g_1(W_{aa}a^{<t-1>} + W_{ax}x^{<t>} + b_a)$ și $y^{<t>} = g_2(W_{ya}a^{<t>} + b_y)$, unde W_{aa} , W_{ax} și W_{ya} sunt ponderile pentru conexiuni (weights) iar b_a și b_y sunt valorile bias. Atât ponderile, cât și valorile bias sunt parametri ce vor fi antrenați.

Rețelele neuronale recurente sunt des utilizate în problemele de procesarea limbajului natural, de speech recognition și chiar de generarea de segmente muzicale, probleme în care ordinea informațiilor are o mare importanță, deoarece iau în calcul aspectul temporal al datelor.

O problemă a acestor tip de rețele este dificultatea accesării informațiilor care se regăsesc la un număr mai mare de pași în trecut deoarece gradientul poate crește sau descrește exponențial odată cu numărul mare de straturi care crește odată cu lungimea datelor de intrare.

LSTM

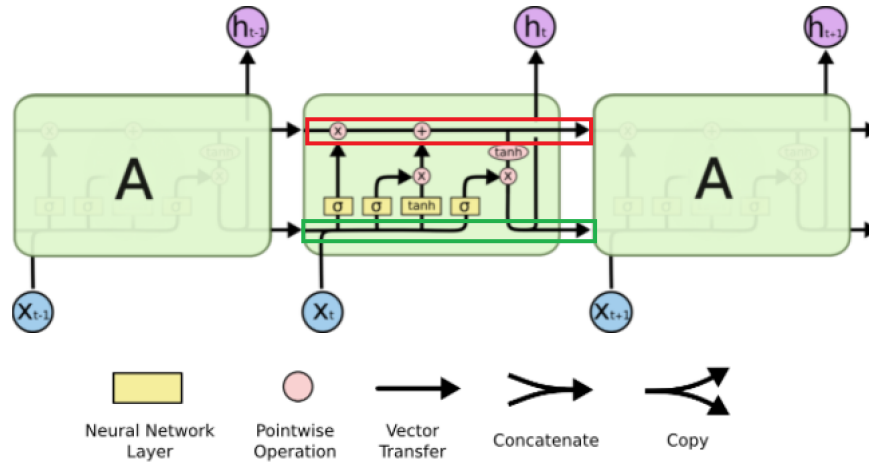
Rețelele Long Short Term Memory (LSTM) sunt un subtip de rețele neuronale recurente care rezolvă problema învățării dependențelor de lungă durată, prin adăugarea unei stări menite să rețină informații pentru perioade mai lungi de timp.

Structura rețelei este similară cu cea a unei rețele recurente tradiționale, diferența

²Recurrent Neural Networks cheatsheet

fiind la nivelul celulei care, acum nu mai conține doar un strat, ci patru straturi care interacționează direct între ele 6.4 ³.

Figura 6.4: Arhitectura LSTM

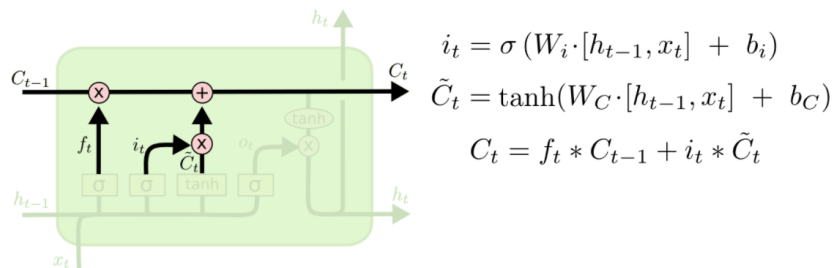


Zona roșie a figurii reprezintă elementele ce țin de memoria de lungă durată a rețelei. Se observă faptul că asupra acestei stări se execută doar două operații. Prima operație este legată de uitarea(eliminarea) informațiilor, iar a doua operație este asociată cu adăugarea de noi informații.

Prima operație este o înmulțire bazată pe elemente (produs Hadamard) dintre starea de scurtă durată curentă și output-ul stratului sigmoid, $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$, unde $[h_{t-1}, x_t]$ reprezintă concatenarea output-ului stratului anterior cu input-ul stării curente. Datorită proprietăților funcției de activare sigmoidă, valorile rezultate aparțin intervalului $[0, 1]$, unde 1 semnifică faptul că informațiile va fi păstrată, iar pentru 0 va fi complet eliminată [22].

A doua operație asupra stării ce ține de memoria de lungă durată este ilustrată în figura 6.5 3.

Figura 6.5: Actualizarea stării celulei



³<http://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Similar cu prima operație efectuată output-ul stratului sigmoid, i_t , va acționa ca un filtru asupra valorilor $\tilde{C}_t$ ce vor fi adunate la starea C_t a celulei. Astfel, se adaugă stării doar informațiile noi ce sunt considerate relevante de rețea.

Zona verde din figura 6.4 reprezintă output-ul celulei. Acest output va fi construit prin aplicarea unei funcții de activare (tangenta hiperbolică) asupra stării celulei și aplicarea unui filtru asupra rezultatului prin înmulțirea cu output-ul stratului sigmoid.

Arhitectura LSTM implementată

Având în vedere contextul temporal al datelor extrase din segmentele audio, a fost construit un model bazat pe LSTM pentru a vedea dacă o procesare ce ține cont de contextul temporal reușește să aibă performanțe mai bune decât o rețea convoluțională, ținând cont de datele problemei abordate.

Modelul construit este format din doua straturi LSTM cu 64 de unități, urmate de un strat dens conectat cu 64 de neuroni, un strat Dropout de 0.3 și un strat dens conectat de output cu activare Softmax și un număr de neuroni egal cu numărul de genuri distinte din setul de date.

6.1.3 CNN-LSTM

Combinarea rețelelor convoluționale cu rețele recurente este des utilizată în practică pentru recunoașterea activităților din videoclipuri sau din secvențe de imagini, dar și pentru recunoașterea sunetelor sau a genurilor muzicale.

Rețelele CNN-LSTM combină elementele de procesare a datelor bidimensionale sau tridimensionale ale rețelelor convoluționale și aspectele temporale surprinse de rețelele LSTM. Straturile convoluționale reduc dimensiunea inputurilor și extrag, în același timp caracteristicile esențiale ale datelor de intrare. Caracteristicile extrase de rețeaua CNN sunt apoi introduse într-o rețea LSTM care procesează evoluția caracteristicilor, generând o predicție.

Arhitectura CNN-LSTM implementată

Rețeaua CNN are 4 straturi convoluționale cu 64 de nuclee de dimensiunea (3, 3). Fiecare strat convoluțional este urmat de un strat de Max Pooling cu dimensiunile (2, 2), (3, 3), (4, 4), (4, 4), și un strat de Batch Normalization și Dropout de 0.1. Output-ul rețelei CNN este aplatizat și introdus ca input în rețeaua LSTM.

Rețeaua LSTM are aceeași arhitectură cu cea implementată în capitolul anterior. În continuare sunt prezentate comparațiile între rețelele implementate.

6.1.4 Comparații

Rețelele CNN și LSTM au fost antrenate pe 20 de coeficienți MFCC pentru segmente de 5 secunde. Dimensiunea datelor de intrare este de 128x20, iar dimensiunea datelor de ieșire corespunde numărului de genuri distincte prezise de rețea, 24 în cazul comparațiilor. Rezultatele antrenării pentru datele de antrenare și validare pot fi observate în figura 6.6 și 6.7. Pe datele de test acuratețea prezicerii pentru CNN și LSTM a fost de 0.3466, respectiv 0.4102.

Figura 6.6: Rezultatele antrenării pentru arhitectura CNN

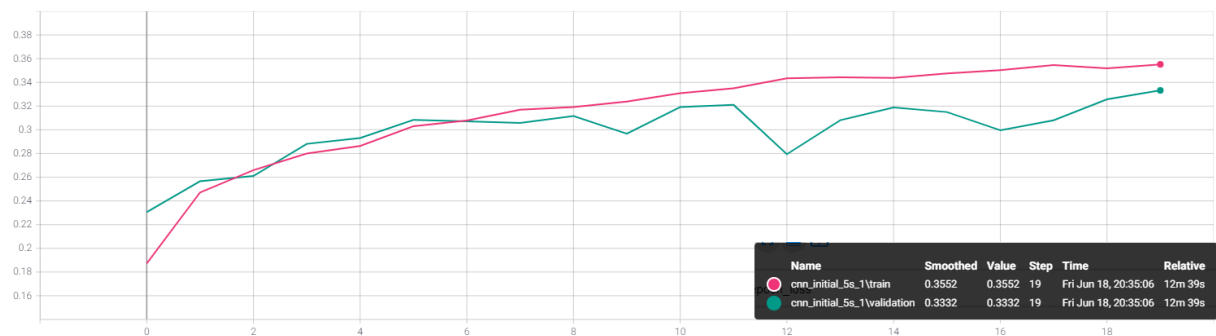
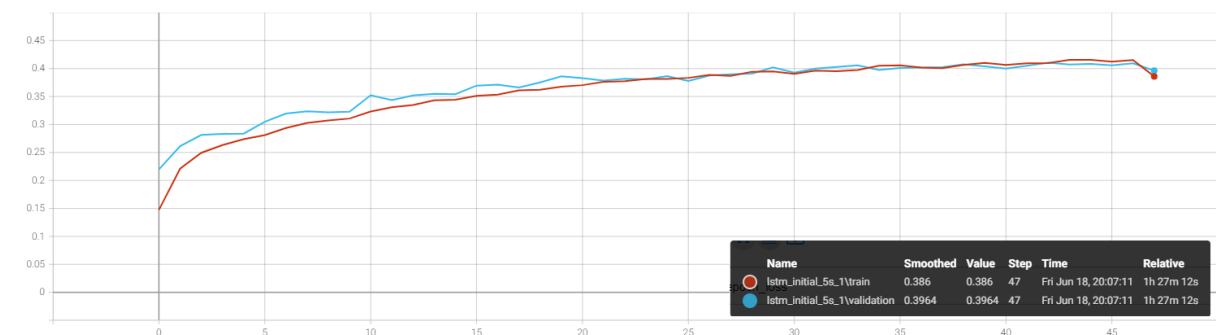


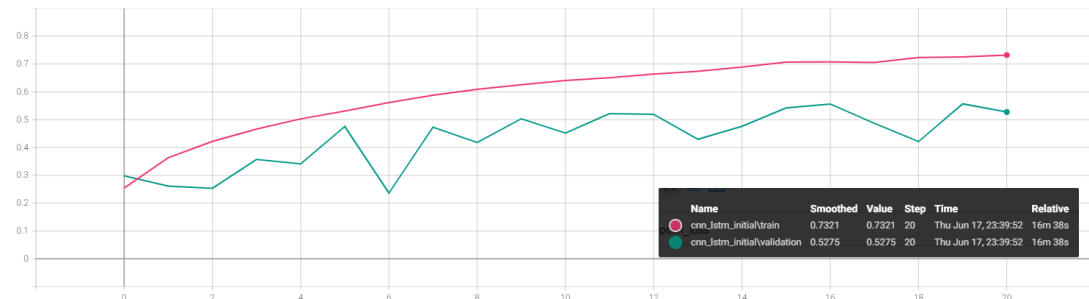
Figura 6.7: Rezultatele antrenării pentru arhitectura LSTM



Având în vedere doar aceste rețele, spectrogramele mel au o dimensiune prea mare pentru a putea fi clasificate, iar segmentele cu lungimi de ordinul sutelor încep să pună probleme rețelei LSTM. În continuare, experimentele pentru rețeaua CNN-LSTM au ca date de input spectrograme Mel extrase din segmente de 15 secunde. Având în vedere faptul că rețeaua CNN reduce dimensiunea datelor de input pentru rețeaua

LSTM, lungimea datelor nu mai este o problema. Rezultatele antrenării pentru datele de antrenare si validare pot fi observate în figura 6.8, iar acuratețea pe datele de test a fost de 0.5124

Figura 6.8: Rezultatele antrenării pentru arhitectura inițială CNN-LSTM



Având în vedere diferența mare dintre acuratețea pe datele de test și acuratețea pe datele de antrenare este evidentă prezența fenomenului de *overfitting*. Pentru a preveni *overfittingul* am adăugat un strat de Dropout de 0.1 după fiecare strat convoluțional, iar rezultatele s-au îmbunătățit considerabil după cum se poate observa în figurile 6.9 și 6.10, acuratețea pe datele de test devenind 0.6140.

Figura 6.9: Rezultatele antrenării pentru arhitectura CNN-LSTM cu Dropout

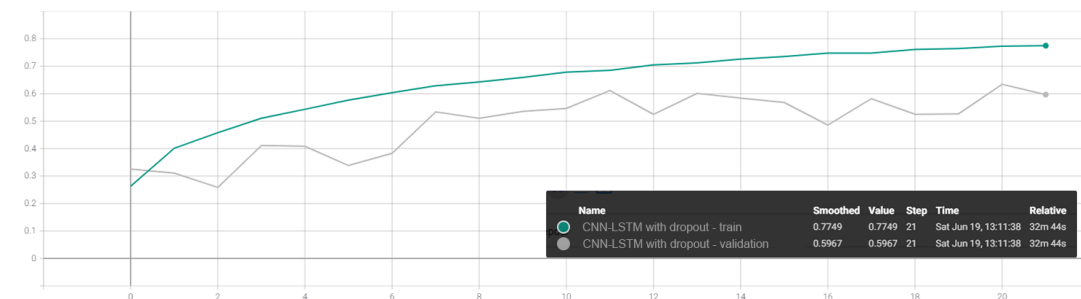
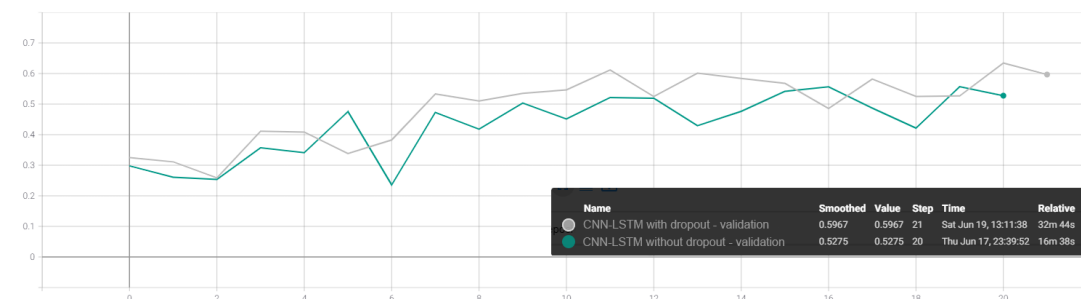


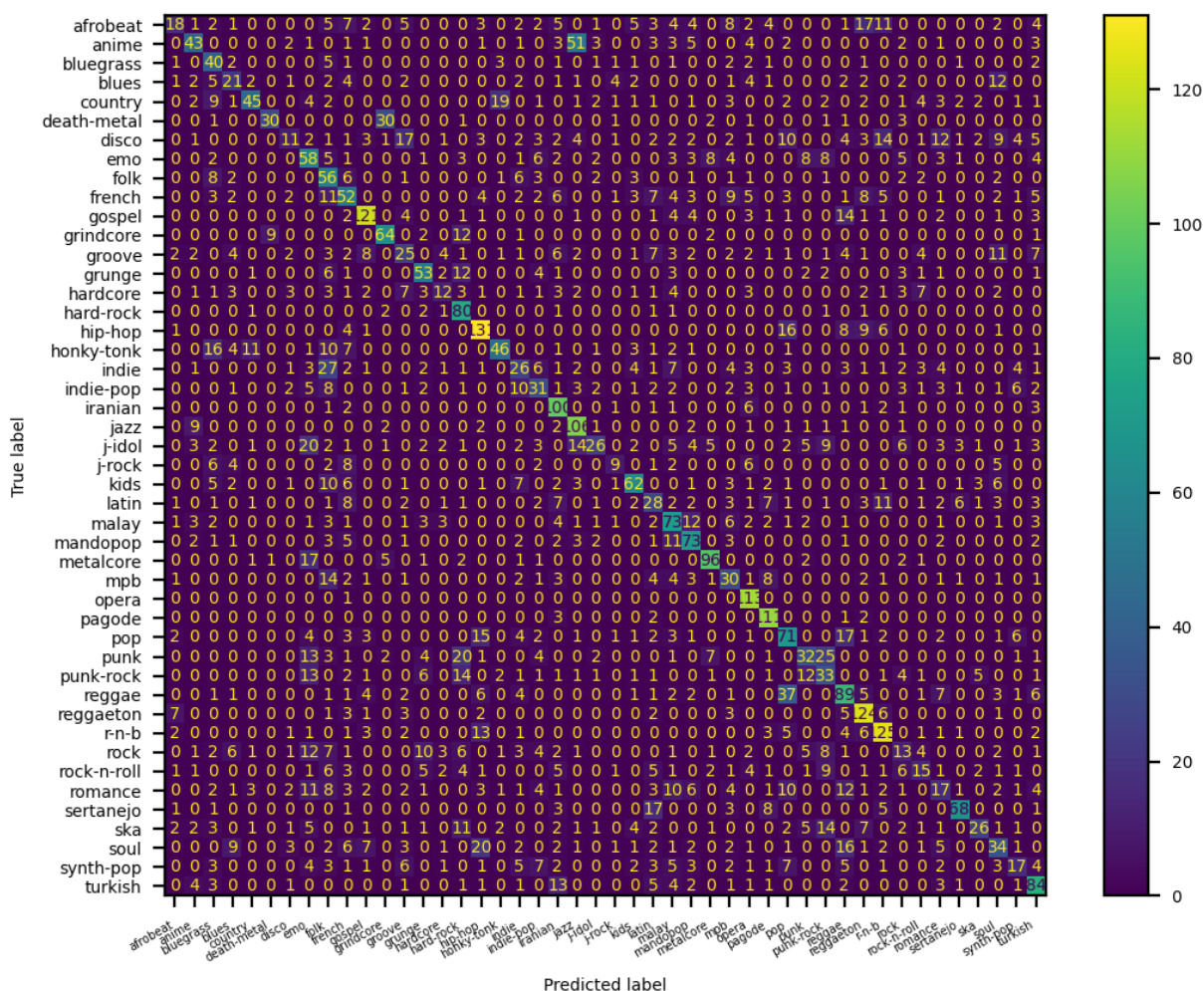
Figura 6.10: Comparația între acuratețea pe datele de validare între CNN-LSTM cu și fără Dropout după straturile convoluționale



6.2 Selecția genurilor

Spotify oferă 126 de genuri muzicale ca *seed* pentru funcționalitatea de recomandări ⁴. Dintre aceste genuri, au fost eliminate genurile care conțin în mare parte doar instrumental precum *house*, *techno* sau *classical* și genurile muzicale care ar introduce ambiguitate precum *sad*, *holiday*, *rainy-day* sau *workout*. După această filtrare inițială s-a ajuns la un set de date cu 46 de genuri muzicale. Rețeaua CNN-LSTM cu Dropout descrisă în subcapitolul anterior a fost antrenată pe acest set de date, iar pentru o selecție ulterioară a genurilor cu scopul de a obține o precizie mai bună a fost construită matricea de confuzie pe setul de date de test 6.11.

Figura 6.11: Matricea de confuzie pentru 46 de genuri muzicale

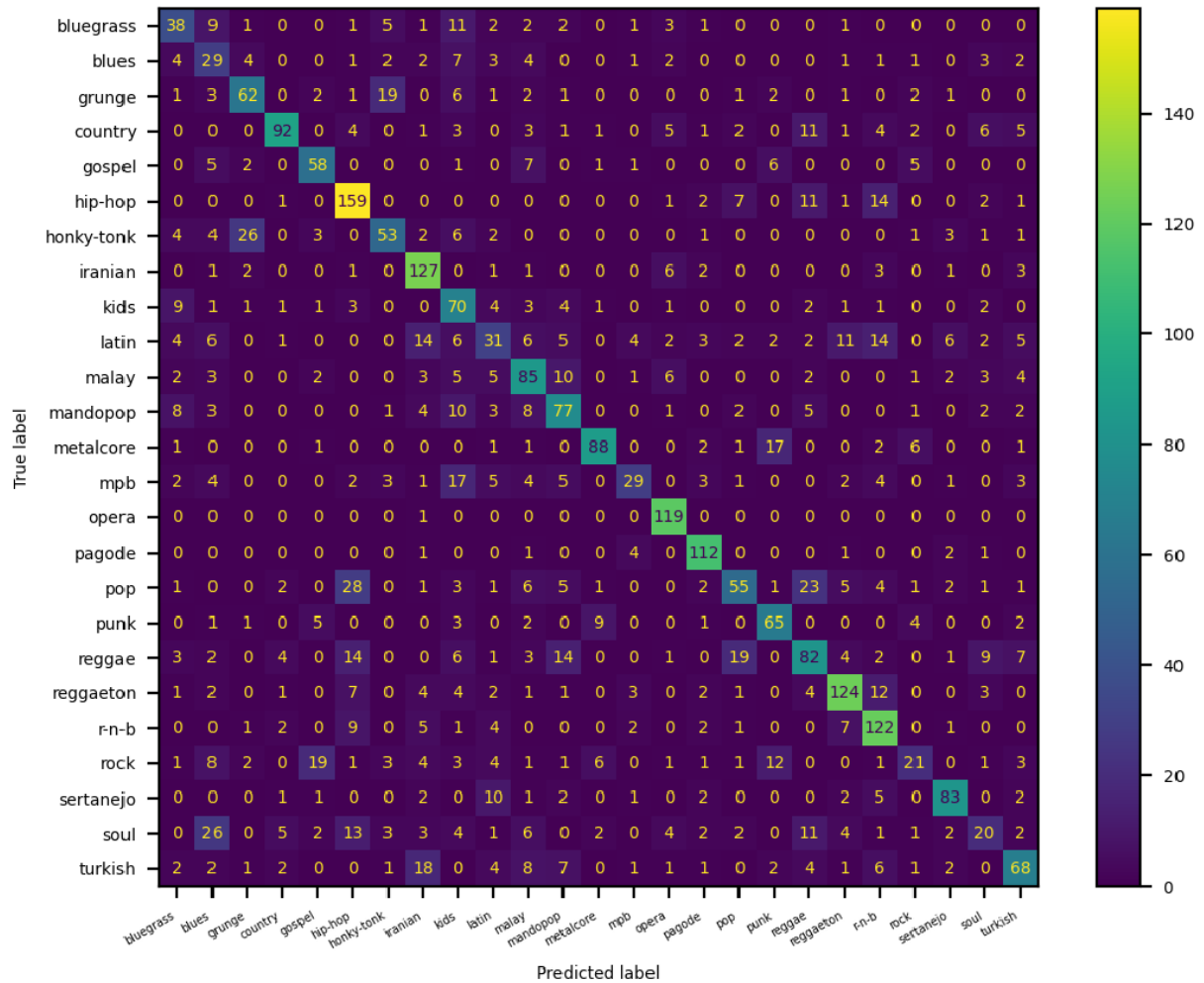


Pe baza acestei matrici s-au eliminat progresiv genurile care erau prezise predominant greșit precum, cu valori mici pe diagonala matricii, *disco*, *synth-pop* sau *j-rock* și

⁴<https://developer.spotify.com/console/get-available-genre-seeds/>

genurile care generau *false positives*, mai multe genuri fiind prezise fals aparținând unui anume gen. Acest lucru se poate observa pe liniile matricii de confuzie. Printre aceste genuri s-au numărat *folk*, *hardrock* sau *emo*. Selecția finală poate fi observată în figura 6.12, genul *kids* fiind eliminat ulterior din cauza problemei descrise în capitolul 7.3.

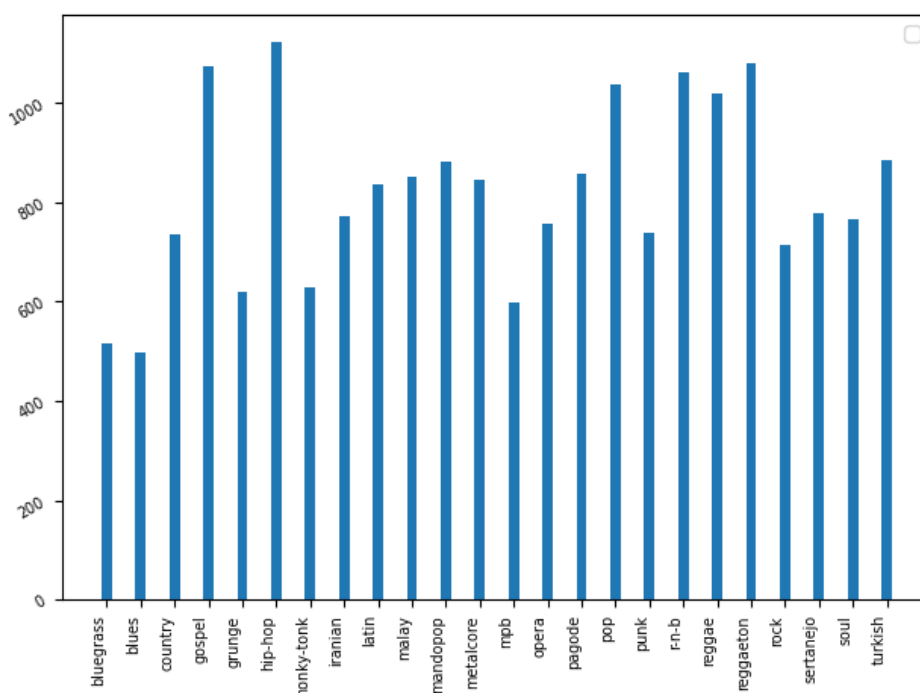
Figura 6.12: Matricea de confuzie pentru 25 de genuri muzicale



6.3 Distribuția datelor

După cum a fost menționat în capitolul 3, după curățarea datelor și eliminarea melodiilor care nu au destulă informație vocală, nu mai există o distribuție egală a genurilor în ceea ce privește numărul de fișiere audio per gen. În plus, numărul de instanțe de antrenare pentru fiecare gen nu este definit de numărul de fișiere audio, ci de numărul de segmente de 15 secunde în care este împărțit fiecare fișier. Astfel, distribuția reală a datelor este reprezentată în figura 6.13

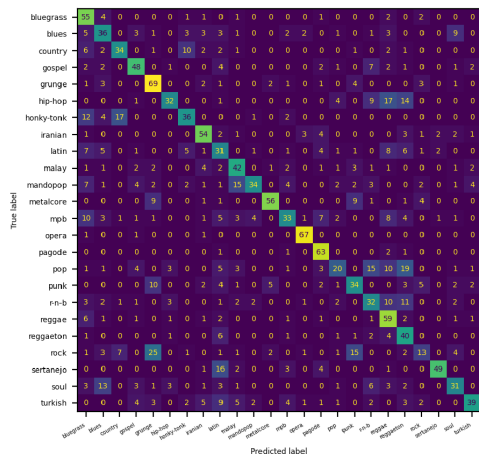
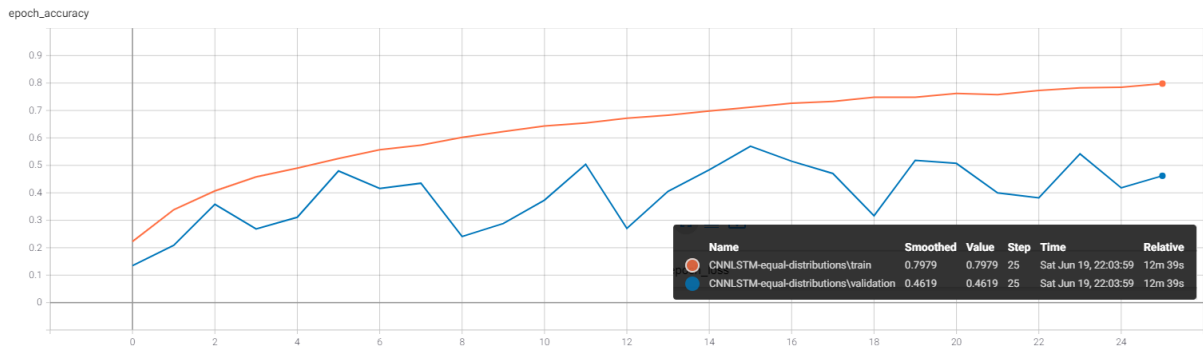
Figura 6.13: Distribuția datelor pe genuri muzicale



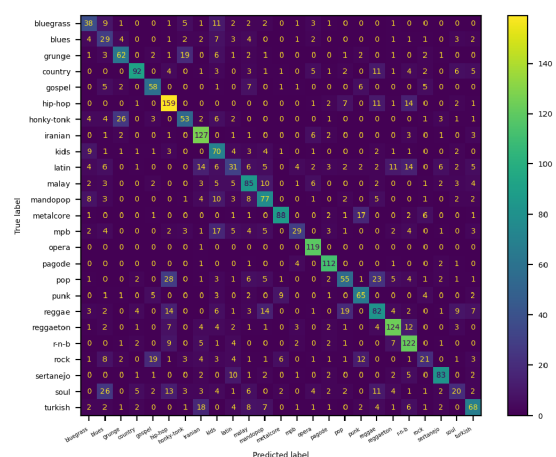
Pentru a rezolva problema distribuției inegale am adăugat în pipeline-ul de încărcare a datelor în memorie opțiunea unui threshold. Astfel, genurile care au mai puține instanțe decât valoarea threshold, nu vor fi luate în considerare, iar pentru genurile care au mai multe instanțe decât valoarea threshold se va alege un subset de date de dimensiunea threshold-ului specificat. Utilizând această opțiune, rețeaua CNN-LSTM a fost antrenată pe un set de date cu 498 de instanțe din fiecare gen (numărul de instanțe pentru genul blues). Rezultatele pe datele de antrenare și validate pot fi observate în figura 6.14. Pe datele de test, acuratețea prezicerii a fost de 0.5616. După cum se poate observa, rezultatele obținute anterior, atunci când este folosit întreg setul de date, sunt mai bune decât atunci când numărul de instanțe este egal pentru toate genurile. O acuratețe mai bună a prezicerii pentru cazul în care datele

nu sunt distribuite egal poate fi observată și în matricele de confuzie din figura 6.15. Un posibil motiv este numărul semnificativ mai mare de date atunci când este folosit întreg setul de date, astfel este evidențiată nevoia unui set mai mare de date.

Figura 6.14: Rezultatele antrenării pentru arhitectura CNN-LSTM cu distribuția egală a datelor



(a) Distribuție egală



(b) Setul întreg de date

Figura 6.15: Comparație între matricele de confuzie

Astfel, modelul final utilizat în sistemul de recomandare a fost antrenat pe întreg setul de date, fără a ține cont de distribuția genurilor.

Capitolul 7

Sistemul de recomandare

Aplicația practică a modelului antrenat este un sistem de recomandare sub forma unei aplicații web care permite utilizatorului să încarce o melodie și să primească ca recomandare alte melodii similare.

Când utilizatorul încarcă un fișier audio acesta este trimis sistemului de recomandare aflat pe serverul de backend. La pornirea serverului este instanțiat un obiect de tipul `PredictionSystem`. Parametrii cu care este instanțiat sunt un rezultat al experimentelor realizate în capitolul precedent. Așadar, se specifică o dimensiune a inputului pentru spectrograme Mel pentru segmente de 15 secunde și o dimensiune a outputului de 24 ce corespunde numărul final de genuri distincte selectate.

Pipeline-ul de procesare a fișierelor primite este similar cu cel descris în capitolele 4 și 5. Fișierul este descărcat local pentru procesare, separat în instrumental și voce, segmentul audio ce conține vocea este împărțit în segmente de 15 secunde, cu offset-uri din secundă în secundă, apoi se extrag spectrogramele Mel și se convertesc în Tensors. Datele obținute sunt introduse în modelul rețelei CNN-LSTM pentru a obține vectorii de predicție.

Deoarece ultimul strat dens conectat are funcția de activare Softmax, fiecare valoare este un procent, reprezentativ pentru apartenența unui segment la genul ce corespunde indexului din vectorul de predicție.

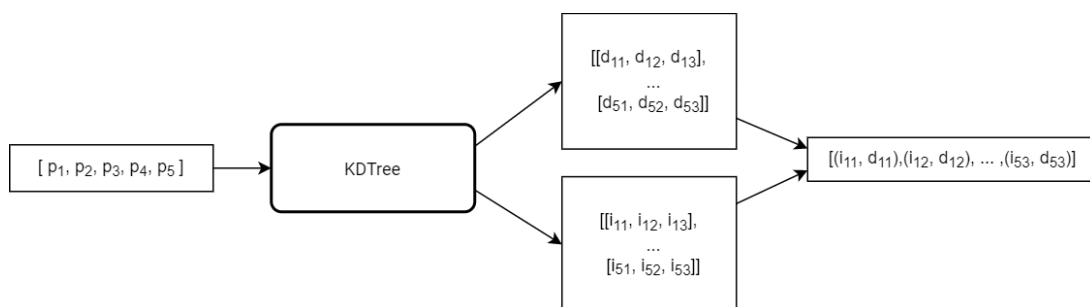
7.1 Recomandare

Toate segmentele din setul de date au fost trecute prin model, iar vectorii de predicție au fost salvați într-un fișier, împreună cu numele segmentului ce conține

genul și id-ul Spotify al melodiei inițiale. La crearea serverului, obiectul Prediction-System instanțiază un obiect KDTree care încarcă vectorii de predicție, pe care îi organizează sub forma unui arbore KDTree, și informațiile adiționale menționate.

Odată ce au fost obținuți vectorii de predicție pentru segmente, sunt căutați cei mai apropiați 3 vecini pentru fiecare vector de predicție împreună cu distanțele dintre vecini și vectorul de predicție. Se selectează cei mai apropiați 12 vecini după procesul ilustrat în figura 7.1, unde p_i este un vector de predicție pentru un segment al melodiei primită ca input, i_{ij} indexul vectorului de predicție a unui vecin existent în setul de date, iar d_{ij} este distanța dintre vectorul de predicție p_i și vecinul cu vectorul de predicție la indexul i_{ij} .

Figura 7.1: Extragerea vecinilor din KDTree



Lista rezultată în figura 7.1 este sortată după distanțele d_{ij} și sunt returnate id-urile ce corespund celor 12 vecini cu cele mai mici distanțe, identificate prin indexul i_{ij} al vecinilor. Acești vecini din setul de date sunt, de fapt, melodiile distincte ce vor fi recomandate, deoarece sunt cele mai similare cu segmentele primite ca input.

7.2 Prezicerea genurilor muzicale

Au fost două abordări în prezicerea genurilor muzicale.

Prima abordare a fost calcularea genului muzical pentru fiecare segment, ca fiind genul cu cel mai mare procent din vectorul de predicție. Astfel, în funcție de numărul de segmente care au fost clasificate ca aparținând unui anumit gen, s-au calculat procente pentru fiecare gen prezis.

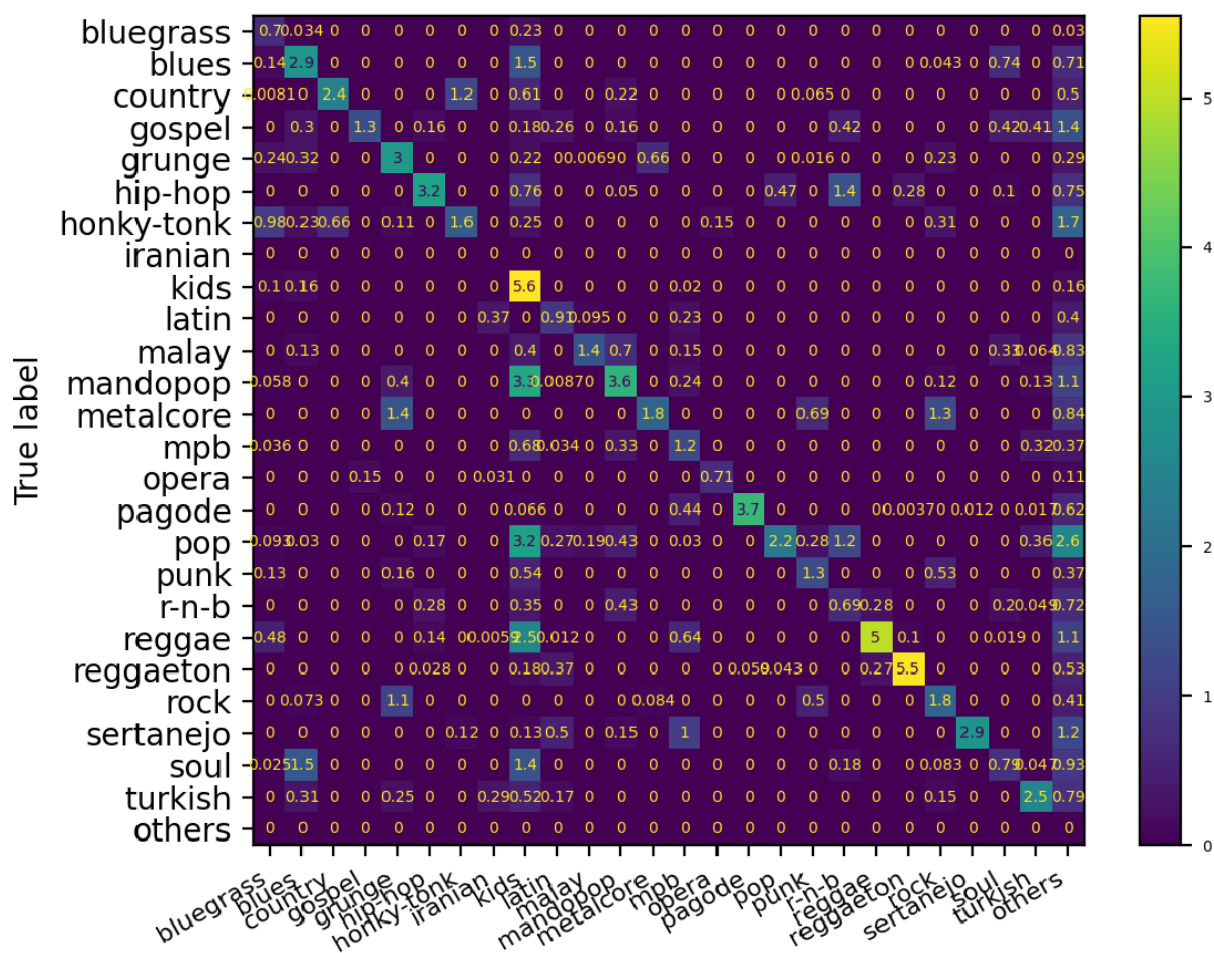
A doua abordare a fost adunarea tuturor vectorilor de predicție obținuți. Se selectează cele mai mari 3 valori din vectorul sumă obținut, corespunzătoare genurilor predominante și se calculează procentul lor ca fiind valoare împărțită la suma valorilor din vector. Restul genurilor sunt incluse în categoria *others*. Chiar dacă rezultatele

sunt similare cu cele obținute utilizând prima metodă, această metodă are performanțe mai bune atunci când diferențele dintre procentele din vectorii de predicție obținuți nu sunt mari.

7.3 Experimente

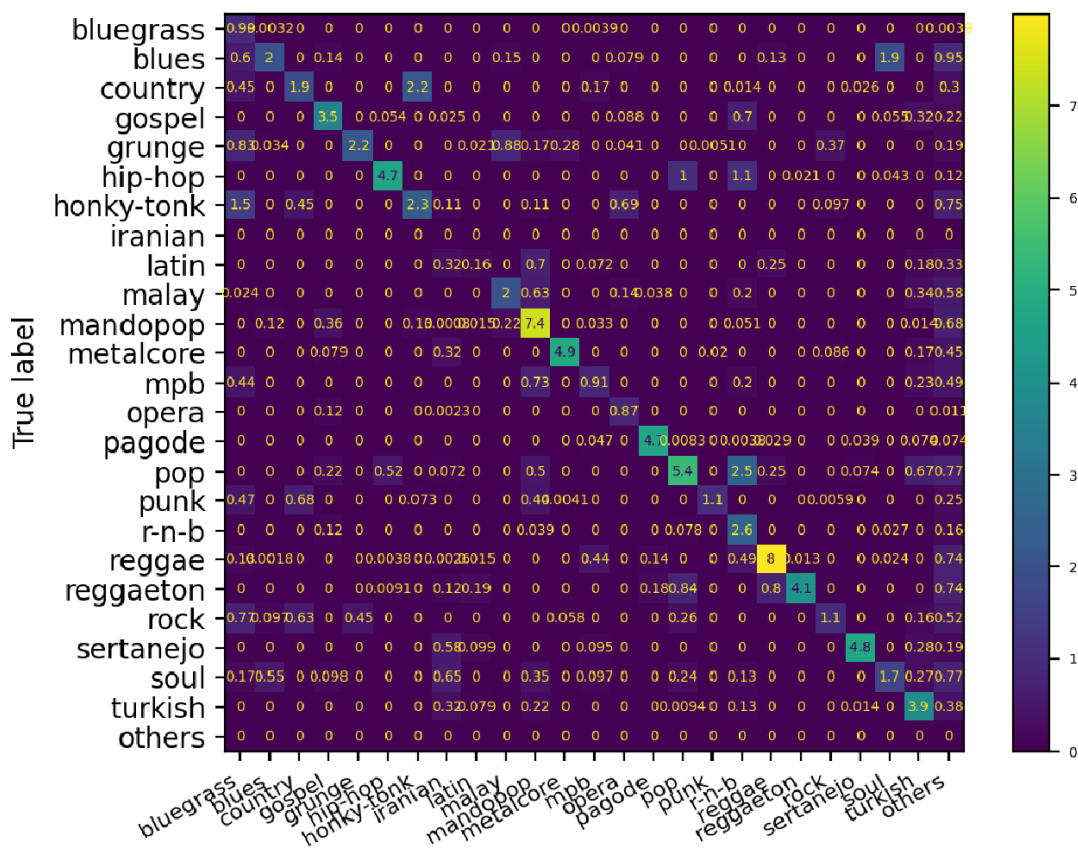
Un prim experiment a fost compararea genurilor prezise de sistem cu cele reale pentru segmente existente în setul de date. A fost observată o problemă în cazul genului *kids*. Un număr mare de segmente erau fals prezise ca aparținând acestui gen după cum se observa în figura 7.2.

Figura 7.2: Genurile prezise inițial de sistem



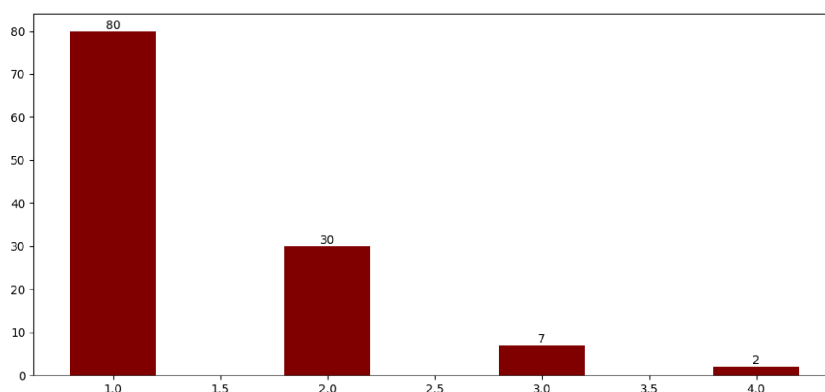
Ca urmare a acestui rezultat, genul *kids* a fost eliminat din setul de date, rețeaua a fost re-antrenată și s-a rulat din nou experimentul. Rezultatele pentru genurile finale selectate sunt ilustrate în figura 4.2

Figura 7.3: Genurile prezise de sistem după eliminarea genului kids



Un alt experiment cu scopul de a identifica abilitatea sistemului de a recomanda melodii similare a fost calcularea melodiilor recomandate care fac parte din același album al unui artist pentru un segment. Rezultatele pot si observate în figura 7.4.

Figura 7.4: Numărul de melodii recomandate din același album



Experimentul a fost rulat pentru 119 segmente existente în setul de date. În 80 de cazuri a fost recomandată doar melodia din setul de date din care face parte segmentul,

în 20 de cazuri printe melodiile recomandate se afla melodia inițială și altă melodie din același album, iar în 9 cazuri, mai mult de 2 melodii din același album au fost recomandate.

Segmentele descărcate utilizând Spotify API au doar 30 de secunde. Un subset de 47 de melodii dintre segmentele utilizate pentru experimentul anterior au fost descărcate în întregime, iar experimentul a fost rulat din nou. Rezultatele pot fi observate în figurile 7.5 și 7.6

Figura 7.5: Genurile prezise de sistem pentru melodiile întregi

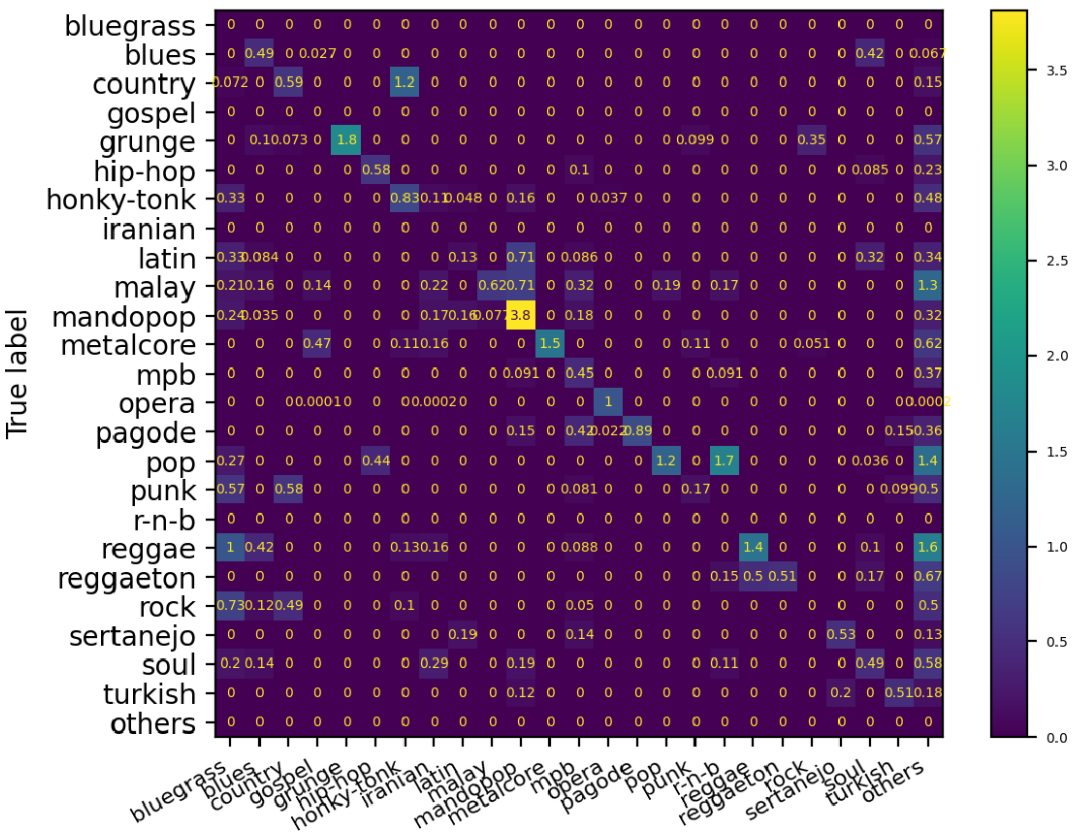
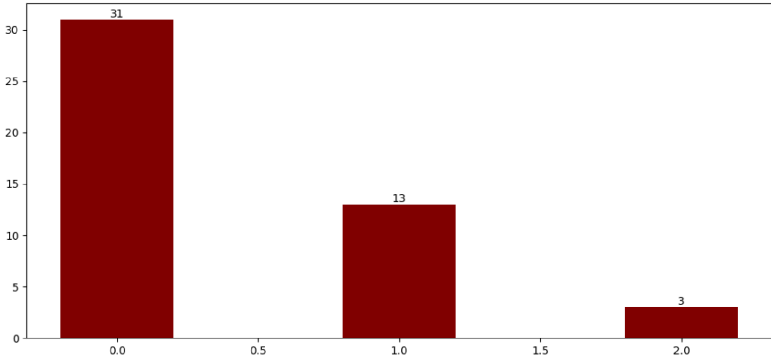


Figura 7.6: Numărul de melodii recomandate din același album pentru melodiile întregi



În 31 dintre cazuri, segmentul existent în setul de date din melodia întreagă nu a fost identificat. Acest lucru este datorat offset-ului utilizat pentru selectarea segmentului, segmentele nu se suprapun perfect peste cel existent în setul de date și se găsesc alte melodii cu segmente mai similare decât cea originală.

Concluzii

7.4 Direcții viitoare

7.4.1 Setul de date

Un prim pas în obținerea unor rezultate mai bune este construirea unui set de date ce conține mai multe date ce conține fișiere audio acapella etichetate manual. Astfel, se reduc zgomotele rezultate în urma separării, se rezolvă problema pierderii elementelor de rimt rezultate în urma eliminării porțiunilor de linște și prezicerea nu se mai bazează pe clasificarea genurilor oferită de Spotify care poate prezenta un factor de instabilitate.

7.4.2 Arhitectura rețelei

Un alt factor care poate fi îmbunătățit este arhitectura rețelei utilizată. În funcție de dimensiunea setului de date, se pot utiliza rețele de dimensiuni mai mare care, aplicate pe datele curente ar converge rapid, ajungând la overfitting.

7.4.3 Noi funcționalități

Având în vedere ideea de recomandare de melodii cu voci similare celei introduse ca input, sistemul de recomandare poate fi extins pentru a lua în calcul mai multe aspecte decât genul în care se încadrează vocea. Aceste noi funcționalități pot include identificarea genului vocii, identificarea range-ului vocal și chiar procesarea discursului pentru a sugera melodii care au un mesaj similar.

Bibliografie

- [1] Lippens, Stefaan, Jean-Pierre Martens, and Tom De Mulder. "A comparison of human and automatic musical genre classification." 2004 IEEE international conference on acoustics, speech, and signal processing. Vol. 4. IEEE, 2004.
- [2] Silla Jr, Carlos N., Alessandro L. Koerich, and Celso AA Kaestner. "Feature selection in automatic music genre classification." 2008 Tenth IEEE International Symposium on Multimedia. IEEE, 2008.
- [3] Hershey, Shawn, et al. "CNN architectures for large-scale audio classification." 2017 IEEE international conference on acoustics, speech and signal processing (icassp). IEEE, 2017.
- [4] Choi, Keunwoo, et al. "Convolutional recurrent neural networks for music classification." 2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2017.
- [5] Ghosal, Soumya Suvra, and Indranil Sarkar. "Novel Approach to Music Genre Classification using Clustering Augmented Learning Method (CALM)." AAAI Spring Symposium: Combining Machine Learning with Knowledge Engineering (1). 2020.
- [6] Rosner, A., Kostek, B. Automatic music genre classification based on musical instrument track separation. J Intell Inf Syst 50, 363–384 (2018).
- [7] Huh, Bryan, and Arun Miduthuri. "VOCAL-BASED MUSICAL GENRE CLASSIFICATION."
- [8] Nasrullah, Zain, and Yue Zhao. "Music artist classification with convolutional recurrent neural networks." 2019 International Joint Conference on Neural Networks (IJCNN). IEEE, 2019.

- [9] Hennequin, Romain, et al. "Spleeter: a fast and efficient music source separation tool with pre-trained models." *Journal of Open Source Software* 5.50 (2020): 2154.
- [10] Prétet, Laure, et al. "Singing voice separation: A study on training data." *ICASSP 2019-2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2019.
- [11] LeCun, Yann, et al. "Gradient-based learning applied to document recognition." *Proceedings of the IEEE* 86.11 (1998): 2278-2324.
- [12] Ioffe, Sergey, and Christian Szegedy. "Batch normalization: Accelerating deep network training by reducing internal covariate shift." *International conference on machine learning*. PMLR, 2015.
- [13] Srivastava, Nitish, et al. "Dropout: a simple way to prevent neural networks from overfitting." *The journal of machine learning research* 15.1 (2014): 1929-1958.
- [14] Kingma, Diederik P., and Jimmy Ba. "Adam: A method for stochastic optimization." *arXiv preprint arXiv:1412.6980* (2014).
- [15] Costa, Yandre MG, Luiz S. Oliveira, and Carlos N. Silla Jr. "An evaluation of convolutional neural networks for music classification using spectrograms." *Applied soft computing* 52 (2017): 28-38.
- [16] He, Kaiming, et al. "Deep residual learning for image recognition." *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016.
- [17] Krizhevsky, Alex, Ilya Sutskever, and Geoffrey E. Hinton. "Imagenet classification with deep convolutional neural networks." *Advances in neural information processing systems* 25 (2012): 1097-1105.
- [18] Jansson, Andreas, et al. "Singing voice separation with deep u-net convolutional networks." (2017).
- [19] Spectrograms
- [20] MFCC features
- [21] Recurrent Neural Networks cheatsheet
- [22] Understanding LSTM Networks

- [23] Spotify API documentation
- [24] Librosa documentation
- [25] Spleeter
- [26] Open-Unmix
- [27] Tensorflow documentation
- [28] tensorflow.data API documentation
- [29] Scikit-Learn documentation
- [30] Flask documentation
- [31] React documentation
- [32] MongoDB Atlas